

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN**



ĐỒ ÁN CUỐI KỲ

Nghiên Cứu Và Xây Dựng Mô Hình Dịch Máy Văn Bản Cổ Từ Tiếng Việt Sang Tiếng Trung

Môn học: Xử Lý Ngôn Ngữ Tự Nhiên

Giáo Viên Hướng Dẫn:

PGS. TS. Đinh Điền
TS. Nguyễn Hồng Bửu Long

Học viên thực hiện:

25C01040 Phạm Thị Ánh Phát
25C23002 Nguyễn Nhật Quang
25C23014 Lê Hoàng Như
25C23018 Đinh Phan Khánh Vũ

Thành Phố Hồ Chí Minh, 2026

Mục lục

1	Giới thiệu	1
1.1	Bối cảnh và động lực nghiên cứu	1
1.1.1	Giới thiệu bài toán dịch máy	1
1.1.2	Đặc thù của dịch máy văn bản cổ	1
1.1.3	Đặc điểm của dữ liệu văn bản cổ	2
1.2	Nêu vấn đề	2
1.3	Mục tiêu nghiên cứu	2
1.4	Câu hỏi nghiên cứu	3
1.5	Đóng góp của đề tài	3
2	Mô tả dữ liệu	3
2.1	Tổng quan	3
2.2	Đặc điểm dữ liệu	4
2.2.1	Quy mô và phân bố độ dài	4
2.2.2	Đơn vị biểu diễn và từ vựng	4
2.2.3	Tính toàn vẹn và mức độ trùng lặp	5
2.3	Tiền xử lý dữ liệu	5
2.4	Hạn chế của dữ liệu	6
3	Phương pháp	7
3.1	Tổng quan pipeline	7
3.2	Dữ liệu và Tiền xử lý	8
3.2.1	Chuẩn hoá văn bản	8
3.2.2	Đọc dữ liệu và kiểm tra toàn vẹn	9
3.2.3	Ba dạng biểu diễn của câu đầu ra	9
3.2.4	Chính sách không dùng subword	10
3.3	Thí nghiệm 1 (E1): Transformer huấn luyện từ đầu	10
3.3.1	Kiến trúc Transformer	10
3.3.2	Fairseq	11
3.3.3	Cấu hình và hàm mục tiêu	11
3.3.4	Giải mã bằng beam search	11
3.3.5	Ví dụ minh hoạ toàn bộ quy trình	12
3.4	Thí nghiệm 2 (E2): Qwen3-8B với QLoRA	13
3.4.1	Mô hình ngôn ngữ lớn và Qwen3-8B	13
3.4.2	LoRA và QLoRA	14
3.4.3	Biểu diễn dữ liệu và hàm mất mát	14
3.4.4	Huấn luyện	14
3.4.5	Giải mã	14
3.4.6	Ví dụ minh hoạ toàn bộ quy trình	15
3.5	Thí nghiệm 3 (E3-custom): Fairseq với truy hồi tri thức nguồn	16
3.5.1	Xây dựng chỉ mục tri thức	16
3.5.2	Quy tắc xếp hạng	16
3.5.3	Kết xuất vào chuỗi nguồn	16
3.5.4	Hệ quả đo được và hạn chế	17

3.5.5	Ví dụ minh hoạ toàn bộ quy trình	17
3.6	Chỉ số đánh giá	18
3.6.1	Chuẩn hoá trước khi chấm điểm	18
3.6.2	BLEU	18
3.6.3	chrF++	19
3.6.4	Diễn giải thang điểm cho các chỉ số	19
3.7	Kiểm định ý nghĩa thống kê	20
4	Thực Nghiệm	21
4.1	Computing Environment	21
4.1.1	Các thông tin về phần cứng	21
4.1.2	Cách setup 3 môi trường cho 3 thí nghiệm khác nhau	21
4.1.3	Seed và determinism	22
4.1.4	Thời gian chạy thực tế	22
4.1.5	12 GB VRAM giải thích phần lớn config	23
4.1.6	Ba chỗ provenance còn hở	23
4.2	Experimental Design	23
4.2.1	So sánh cái gì với cái gì	23
4.2.2	Ba split, và bằng chứng ba hệ thống dùng chung một corpus	23
4.2.3	Tóm tắt ba setup	24
4.2.4	Validation trước, freeze, rồi mới mở test	24
4.2.5	Chấm điểm và kiểm định thống kê	25
4.2.6	E2: 3 epoch hay 5 epoch?	25
4.2.7	Những gì không nằm trong thực nghiệm này	26
4.3	Evaluation Metrics (Chỉ số đánh giá)	26
4.3.1	4.3.1 SacreBLEU	26
4.3.2	4.3.2 chrF++	26
4.3.3	4.3.3 COMET (tùy chọn)	27
4.4	Quy trình đánh giá	27
5	Kết quả	28
5.1	Tổng quan	28
5.2	Kết quả thực nghiệm E1 (Fairseq Transformer)	28
5.3	Kết quả thực nghiệm E2 (Qwen38B QLoRa)	29
5.4	So sánh đối chiếu E1 vs E2 (E1 vs E2 Comparison)	29
5.5	Phân tích ví dụ định tính (Qualitative Examples)	30
6	Error Analysis	31
6.1	Phương pháp phân tích (Error Analysis Methodology)	31
6.2	Hệ thống phân loại lỗi (Error Taxonomy)	31
6.3	Phân bố lỗi (Error Distribution)	31
6.4	So sánh đối chiếu lỗi (Error Comparison)	32
6.5	Hiện tượng Ảo giác (Hallucination)	33
6.6	Đề xuất	33

7	Thảo luận, Hạn chế, kết luận	33
7.1	Thảo luận	33
7.2	Hạn chế	34
7.3	Kết luận	34

1 Giới thiệu

1.1 Bối cảnh và động lực nghiên cứu

1.1.1 Giới thiệu bài toán dịch máy

Dịch máy (Machine Translation–MT) là bài toán tự động chuyển một chuỗi văn bản từ ngôn ngữ nguồn sang ngôn ngữ đích, sao cho vừa bảo toàn nội dung, vừa tạo ra câu phù hợp với từ vựng và cú pháp của ngôn ngữ đích. Với sự phát triển của học sâu, dịch máy nơ-ron (Neural Machine Translation–NMT) mô hình hóa trực tiếp xác suất của câu đích theo câu nguồn. Kiến trúc Transformer, dựa trên cơ chế attention, đã trở thành nền tảng phổ biến của NMT nhờ khả năng mô hình hóa quan hệ phụ thuộc xa và huấn luyện song song hiệu quả [9].

Tuy nhiên, hiệu quả của NMT phụ thuộc đáng kể vào quy mô và chất lượng của ngữ liệu song song. Trong bối cảnh tài nguyên thấp, mô hình huấn luyện từ đầu dễ quá khớp và khó học được các ánh xạ hiếm. Nghiên cứu của Atrio và Popescu-Belis cho thấy cấu hình phù hợp với dữ liệu lớn không thể được áp dụng máy móc cho dữ liệu nhỏ; cụ thể, batch nhỏ có thể tạo hiệu ứng điều chuẩn và giúp Transformer tổng quát hóa tốt hơn [1]. Vì vậy, bài toán của đề tài không chỉ là lựa chọn một kiến trúc dịch, mà còn là xây dựng một quy trình thực nghiệm phù hợp với miền dữ liệu chuyên biệt và có quy mô hạn chế.

Trong nghiên cứu này, hướng dịch được khảo sát là tiếng Việt sang Hán văn cổ (Việt → Hán cổ). Hai cách tiếp cận được đối chiếu gồm: (i) mô hình Transformer NMT huấn luyện trên ngữ liệu của đề tài bằng Fairseq; và (ii) mô hình ngôn ngữ lớn đã tiền huấn luyện, sau đó thích nghi miền bằng QLoRA. Sự đối chiếu này cho phép đánh giá khác biệt giữa việc học ánh xạ dịch từ đầu và việc tận dụng tri thức ngôn ngữ có sẵn của mô hình nền tảng.

1.1.2 Đặc thù của dịch máy văn bản cổ

Trong phạm vi đề tài, “văn bản cổ” được xác định theo miền nội dung: văn bản có nguồn gốc lịch sử hoặc cổ điển, sử dụng lối diễn đạt văn chương, đồng thời chứa nhiều thực thể, chức danh, địa danh, sự kiện, niên hiệu và biểu thức văn hóa đặc thù [2]. Khái niệm này không đồng nhất với việc cả hai phía của cặp câu đều phải sử dụng ngôn ngữ cổ. Cụ thể, câu nguồn trong bộ dữ liệu là bản diễn giải tiếng Việt tương đối hiện đại, còn câu đích là Hán văn cổ trích từ *Đại Việt sử ký toàn thư*. So với dịch văn bản hiện đại, dịch sang Hán văn cổ đặt ra ba khó khăn chính. Thứ nhất, Hán văn cổ có tính hàm súc cao, thường lược bỏ chủ ngữ, đại từ và các thành phần có thể suy ra từ ngữ cảnh; do đó, quan hệ giữa câu tiếng Việt và câu Hán không phải lúc nào cũng là ánh xạ từng từ. Thứ hai, một âm đọc hoặc một tên riêng tiếng Việt có thể tương ứng với nhiều Hán tự, trong khi việc chọn sai một ký tự có thể làm thay đổi thực thể hoặc ý nghĩa lịch sử. Thứ ba, các chức quan, địa danh cổ, niên hiệu, con số và mốc thời gian xuất hiện không đồng đều, khiến nhiều đơn vị quan trọng có tần suất rất thấp trong tập huấn luyện. Mô hình vì thế phải đồng thời học phép chuyển đổi ngôn ngữ, quy ước biểu đạt cổ điển và tri thức đặc thù của sử liệu.

Những đặc điểm trên cũng làm cho đánh giá tự động trở nên khó khăn. Một bản dịch có thể khác bề mặt so với câu tham chiếu nhưng vẫn gần nghĩa; ngược lại, chỉ một Hán tự sai trong tên người, địa danh hoặc niên hiệu đã có thể tạo ra lỗi nghiêm trọng dù điểm tương đồng toàn câu còn cao. Do đó, SacreBLEU và chrF++ cần được sử dụng kết hợp với phân tích lỗi thủ công, đặc biệt đối với lỗi thực thể, số liệu–thời gian, bỏ sót và tự bổ sung thông tin.

1.1.3 Đặc điểm của dữ liệu văn bản cổ

Ngữ liệu cốt lõi của đề tài được xây dựng từ *Đại Việt sử ký toàn thư* và đã được cố định thành ba tập. Tập huấn luyện gồm 19.218 cặp câu được tự động căn chỉnh; tập validation và tập test mỗi tập gồm 510 cặp câu được căn chỉnh thủ công để dùng làm dữ liệu chuẩn đánh giá. Cách tổ chức này tạo ra sự khác biệt về độ tin cậy giữa dữ liệu huấn luyện dạng *silver* và dữ liệu đánh giá dạng *gold*, đồng thời yêu cầu mô hình phải chống chịu được nhiều căn chỉnh trong quá trình học.

Dữ liệu đã được chuẩn hóa về chữ thường ở phía tiếng Việt và loại bỏ dấu câu ở cả hai phía. Ở phía Hán văn, các Hán tự được phân tách bằng khoảng trắng; độ dài trung bình lần lượt xấp xỉ 18,62 Hán tự và 20,92 từ tiếng Việt trên mỗi cặp huấn luyện. Ngoài chênh lệch độ dài giữa hai ngôn ngữ, tập huấn luyện còn có các câu rất ngắn, các câu dài bất thường và một số dòng trùng lặp. Những yếu tố này có thể ảnh hưởng đến phân phối batch, quá trình học căn chỉnh và khả năng tổng quát hóa.

Bên cạnh ngữ liệu song song, gói dữ liệu còn cung cấp kho tri thức có cấu trúc cho khoảng 16.000 Hán/Nôm tự, bao gồm thông tin về âm đọc, nghĩa, cấu tạo và ví dụ. Nguồn tri thức này có tiềm năng hỗ trợ xử lý ký tự hiếm và các ánh xạ Việt–Hán mơ hồ, nhưng phải được xem là nguồn bổ trợ, được tách biệt khỏi thí nghiệm cốt lõi và không được làm rò rỉ thông tin từ tập validation hoặc test. Ngoài ra, do đây là tài sản nghiên cứu của phòng thí nghiệm, ngữ liệu cốt lõi không được công bố hoặc phân phối lại công khai [2].

1.2 Nêu vấn đề

Bài toán trung tâm của đề tài là xây dựng hệ thống dịch từ tiếng Việt sang Hán văn cổ trong điều kiện dữ liệu song song ít, dữ liệu huấn luyện có thể chứa nhiều căn chỉnh và miền đích có nhiều ký tự, thực thể lịch sử cùng cấu trúc diễn đạt hiếm. Một Transformer huấn luyện từ đầu có ưu điểm là quy trình rõ ràng và dễ kiểm soát, nhưng có thể thiếu tri thức để xử lý các trường hợp ít xuất hiện. Ngược lại, mô hình ngôn ngữ lớn có tri thức tiền huấn luyện rộng hơn, song vẫn cần thích nghi miền, kiểm soát đầu ra và hạn chế việc sinh thêm thông tin không có trong câu nguồn.

Từ đó, nghiên cứu cần giải quyết đồng thời bốn vấn đề: thiết lập một baseline NMT có thể tái lập; thích nghi một mô hình ngôn ngữ lớn cho miền Hán văn cổ; so sánh hai hướng tiếp cận bằng các phép đo nhất quán; và xác định những loại lỗi mà điểm số tổng hợp chưa phản ánh đầy đủ. Việc lựa chọn mô hình không chỉ dựa trên BLEU cao hơn, mà còn phải xét khả năng bảo toàn thực thể, số liệu, mốc thời gian, mức độ đầy đủ của bản dịch và nguy cơ sinh nội dung không được hỗ trợ bởi câu nguồn.

1.3 Mục tiêu nghiên cứu

Đề tài hướng đến các mục tiêu sau:

1. Xây dựng baseline Transformer bằng Fairseq cho hướng dịch Việt → Hán cổ với cấu hình huấn luyện, giải mã và checkpoint có thể tái lập.
2. Thử nghiệm mô hình ngôn ngữ lớn Qwen3-8B-Instruct cho miền văn bản cổ bằng QLoRA 4-bit, qua đó tận dụng tri thức tiền huấn luyện trong giới hạn tài nguyên tính toán của đề tài.
3. So sánh công bằng baseline NMT và mô hình ngôn ngữ lớn trên cùng các phân hoạch dữ liệu đã cố định, sử dụng SacreBLEU và chrF++ với phiên bản và cấu hình được ghi nhận đầy đủ.

4. Phân tích đầu ra theo các nhóm lỗi gồm dịch sai, bỏ sót, tự bổ sung thông tin, lỗi thực thể và lỗi số liệu–thời gian; từ đó làm rõ ưu, nhược điểm của từng cách tiếp cận.
5. Khảo sát khả năng khai thác kho tri thức ký tự như một thí nghiệm bổ sung, đồng thời tuân thủ nguyên tắc nguồn gốc dữ liệu và phòng tránh rò rỉ train–validation–test.

1.4 Câu hỏi nghiên cứu

Nghiên cứu tập trung trả lời các câu hỏi sau:

- RQ1.** Trong điều kiện ngữ liệu *Đại Việt sử ký toàn thư* có quy mô hạn chế, Transformer huấn luyện từ đầu đạt chất lượng dịch Việt $\rightarrow$ Hán cổ ở mức nào?
- RQ2.** Việc thích nghi Qwen3-8B-Instruct bằng QLoRA cải thiện SacreBLEU và chrF++ như thế nào so với baseline Transformer khi hai mô hình được đánh giá trên cùng tập test đã cố định?
- RQ3.** Hai mô hình khác nhau ra sao về mức độ bảo toàn nội dung và phân bố các lỗi dịch sai, bỏ sót, tự bổ sung, thực thể và số liệu–thời gian?
- RQ4.** Nguồn tri thức ký tự bổ trợ có giúp cải thiện khả năng xử lý Hán tự hiếm và ánh xạ Việt–Hán mơ hồ so với baseline không sử dụng tri thức hay không?

1.5 Đóng góp của đề tài

Các đóng góp chính của đề tài gồm:

- Hệ thống hóa các thách thức của dịch tiếng Việt sang Hán văn cổ trên ngữ liệu sử học, nhấn mạnh sự khác biệt giữa miền nội dung cổ và hình thức ngôn ngữ cổ.
- Xây dựng quy trình thực nghiệm có thể tái lập cho hai hướng tiếp cận: Transformer NMT huấn luyện từ đầu và mô hình ngôn ngữ lớn thích nghi miền bằng QLoRA.
- Cung cấp phép so sánh định lượng trên các tập dữ liệu đã cố định, kết hợp đánh giá ở cấp độ n-gram, ký tự và kiểm định thống kê để hạn chế kết luận dựa trên chênh lệch điểm ngẫu nhiên.
- Thiết lập khung phân tích lỗi theo đặc thù sử liệu, trong đó tách biệt lỗi dịch thông thường với việc mô hình tự bổ sung thông tin, đồng thời chú trọng lỗi thực thể và số liệu–thời gian.
- Khảo sát vai trò của tri thức ký tự bổ trợ đối với bài toán tài nguyên thấp và lưu trữ đầy đủ cấu hình, dự đoán, chỉ số cùng checkpoint để hỗ trợ tái sử dụng trong các nghiên cứu tiếp theo.

2 Mô tả dữ liệu

2.1 Tổng quan

Ngữ liệu cốt lõi của đề tài là tập song ngữ tiếng Việt–Hán văn cổ được xây dựng từ *Đại Việt sử ký toàn thư*. Trong mỗi cặp dữ liệu, phía nguồn là câu tiếng Việt và phía đích là câu Hán văn cổ

tương ứng. Đây là nguồn dữ liệu bắt buộc cho các thí nghiệm chính của hướng dịch Việt → Hán cổ; ba phân hoạch train, validation và test đã được cố định trước khi thực nghiệm nhằm bảo đảm khả năng so sánh giữa các mô hình và hạn chế việc điều chỉnh mô hình theo tập test [2].

Gói dữ liệu gồm hai thành phần. Thành phần thứ nhất là ngữ liệu song song với tổng cộng 20.238 cặp câu, trong đó có 19.218 cặp train, 510 cặp validation và 510 cặp test. Thành phần thứ hai là tệp `knowledge.json`, một kho tri thức có cấu trúc cho khoảng 16.000 Hán/Nôm tự, chứa các trường thông tin như mã Unicode, âm đọc, biến thể truy vấn, cấu tạo chữ, nghĩa và ví dụ. Ngữ liệu song song được sử dụng trong các thí nghiệm cốt lõi E1 và E2, còn kho tri thức chỉ đóng vai trò dữ liệu hỗ trợ cho thí nghiệm mở rộng.

Theo quy ước về nguồn gốc dữ liệu, tập train được tạo bằng phương pháp tự động căn chỉnh nên được xem là dữ liệu *silver*. Ngược lại, validation và test được căn chỉnh thủ công và đóng vai trò dữ liệu *gold* để lựa chọn mô hình và đánh giá cuối cùng. Sự phân biệt này đặc biệt quan trọng vì số lượng lớn hơn của tập train không đồng nghĩa với độ chính xác căn chỉnh cao hơn các tập đánh giá.

Bang 1: Tổng quan các phân hoạch của ngữ liệu song song

Phân hoạch	Số cặp	Cặp duy nhất	TB từ Việt	TB Hán tự
Train	19.218	17.622	20,92	18,62
Validation	510	510	20,32	17,47
Test	510	510	20,66	17,61
Tổng	20.238	—	—	—

2.2 Đặc điểm dữ liệu

2.2.1 Quy mô và phân bố độ dài

Bảng 1 cho thấy độ dài trung bình của ba phân hoạch tương đối đồng nhất: mỗi câu tiếng Việt có khoảng 20–21 từ, trong khi câu đích có khoảng 17–19 Hán tự. Trung vị độ dài của phía tiếng Việt là 17 từ ở cả ba tập; trung vị phía Hán là 15 Hán tự đối với train và 14,5 Hán tự đối với validation và test. Nhìn chung, câu tiếng Việt dài hơn câu Hán, phù hợp với tính hàm súc và khả năng lược bỏ thành phần của Hán văn cổ.

Mặc dù giá trị trung bình giữa các tập khá ổn định, miền độ dài của tập train rất rộng: từ 1 đến 199 từ tiếng Việt và từ 1 đến 196 Hán tự. Trong khi đó, validation giới hạn ở 3–70 từ tiếng Việt và 2–66 Hán tự; test giới hạn ở 2–90 từ tiếng Việt và 2–79 Hán tự. Các trường hợp quá ngắn hoặc quá dài trong tập train có thể tạo batch mất cân bằng và làm tăng nhiễu trong quá trình học.

2.2.2 Đơn vị biểu diễn và từ vựng

Phía tiếng Việt được biểu diễn dưới dạng chuỗi từ phân cách bằng khoảng trắng và đã được chuyển về chữ thường. Phía Hán văn được tách ở cấp độ ký tự: mỗi token quan sát được đều là một Hán tự đơn và các ký tự được phân cách bằng khoảng trắng. Do đó, khái niệm “độ dài câu đích” và “kích thước từ vựng đích” trong báo cáo lần lượt chỉ số Hán tự và số loại Hán tự, không phải số từ theo cách phân đoạn từ vựng tiếng Trung hiện đại.

Tập train có 401.997 token tiếng Việt thuộc 3.607 loại từ và 357.806 token Hán thuộc 5.386 loại ký tự. Trong số đó, 507 loại từ tiếng Việt và 1.137 Hán tự chỉ xuất hiện đúng một lần. Phân bố đuôi

dài này cho thấy phía đích có độ phân tán từ vựng cao hơn: nhiều Hán tự hiếm, tên riêng, chức danh, địa danh và niên hiệu chỉ cung cấp rất ít tín hiệu học. Đây là một nguyên nhân khiến mô hình huấn luyện từ đầu dễ nhầm lẫn hoặc không sinh đúng các ký tự ít gặp.

2.2.3 Tính toàn vẹn và mức độ trùng lặp

Số dòng ở hai phía của từng phân hoạch hoàn toàn bằng nhau và không có dòng rỗng. Kiểm tra khớp chính xác cũng không phát hiện câu tiếng Việt, câu Hán hay cặp song ngữ nào xuất hiện đồng thời giữa train, validation và test. Kết quả này cho thấy không có rò rỉ trực tiếp do sao chép nguyên dòng giữa các phân hoạch.

Tuy nhiên, trong 19.218 dòng train chỉ có 17.622 cặp song ngữ duy nhất; 1.596 dòng còn lại là các lần lặp, tương đương khoảng 8,3% tập huấn luyện. Một số biểu thức sử ký ngắn và có tính công thức xuất hiện nhiều lần. Sự lặp lại có thể phản ánh đúng đặc trưng của văn bản biên niên, nhưng đồng thời làm tăng trọng số của một số mẫu khi tối ưu mô hình và khiến quy mô dữ liệu hiệu dụng thấp hơn số dòng danh nghĩa.

2.3 Tiền xử lý dữ liệu

Dữ liệu cung cấp đã trải qua các bước căn chỉnh và làm sạch cơ bản trước khi được sử dụng trong đề tài. Quy trình thể hiện qua cấu trúc và nội dung tệp gồm các bước chính sau:

1. **Căn chỉnh song ngữ:** mỗi câu tiếng Việt được ghép với một câu Hán văn tương ứng và lưu ở cùng số dòng trong hai tệp. Tập train sử dụng kết quả tự động căn chỉnh, còn validation và test sử dụng các cặp đã được căn chỉnh thủ công.
2. **Phân chia và cố định dữ liệu:** các cặp câu được chia sẵn thành train, validation và test. Nhóm giữ nguyên các phân hoạch này trong toàn bộ thí nghiệm; validation được dùng để lựa chọn checkpoint và cấu hình, còn test chỉ được mở để đánh giá sau khi lựa chọn mô hình đã được đóng băng.
3. **Chuẩn hóa bề mặt:** toàn bộ câu tiếng Việt được chuyển về chữ thường; dấu câu đã được loại bỏ ở cả hai phía. Các khoảng trắng được dùng làm dấu phân cách token và không có dòng rỗng trong sáu tệp song song.
4. **Tách Hán tự:** phía Hán văn được chèn khoảng trắng giữa từng ký tự. Cách biểu diễn này tạo điều kiện cho việc xây dựng từ điển, huấn luyện và đánh giá ở cấp độ ký tự, đồng thời giảm phụ thuộc vào một công cụ phân đoạn từ dành cho tiếng Trung hiện đại.
5. **Kiểm tra toàn vẹn:** trước khi huấn luyện, số dòng hai phía được đối chiếu, các cặp trùng chính xác được thống kê và giao nhau giữa các phân hoạch được kiểm tra. Không thực hiện khử trùng lặp trên tập train trong thí nghiệm chính nhằm giữ nguyên phiên bản dữ liệu đã được cung cấp.
6. **Chuẩn bị theo mô hình:** đối với Fairseq, dữ liệu tiếp tục được mã hóa theo từ điển hoặc đơn vị dưới từ và chuyển sang định dạng nhị phân phục vụ huấn luyện. Đối với Qwen3-8B-Instruct, mỗi cặp được đóng gói thành mẫu chỉ dẫn gồm câu nguồn và câu trả lời đích; loss chỉ được tính trên phần câu trả lời. Các biến đổi này chỉ được học hoặc thiết lập từ tập train để tránh rò rỉ thông tin đánh giá.

Kho tri thức ký tự được quản lý tách biệt với ngữ liệu song song. Nếu sử dụng trong thí nghiệm bổ sung, mỗi mục tri thức phải được truy xuất từ ký tự hoặc nội dung câu nguồn theo một quy tắc xác định trước; tuyệt đối không sử dụng câu tham chiếu của validation hoặc test để lựa chọn tri thức, sinh thêm dữ liệu huấn luyện hay điều chỉnh đầu ra.

2.4 Hạn chế của dữ liệu

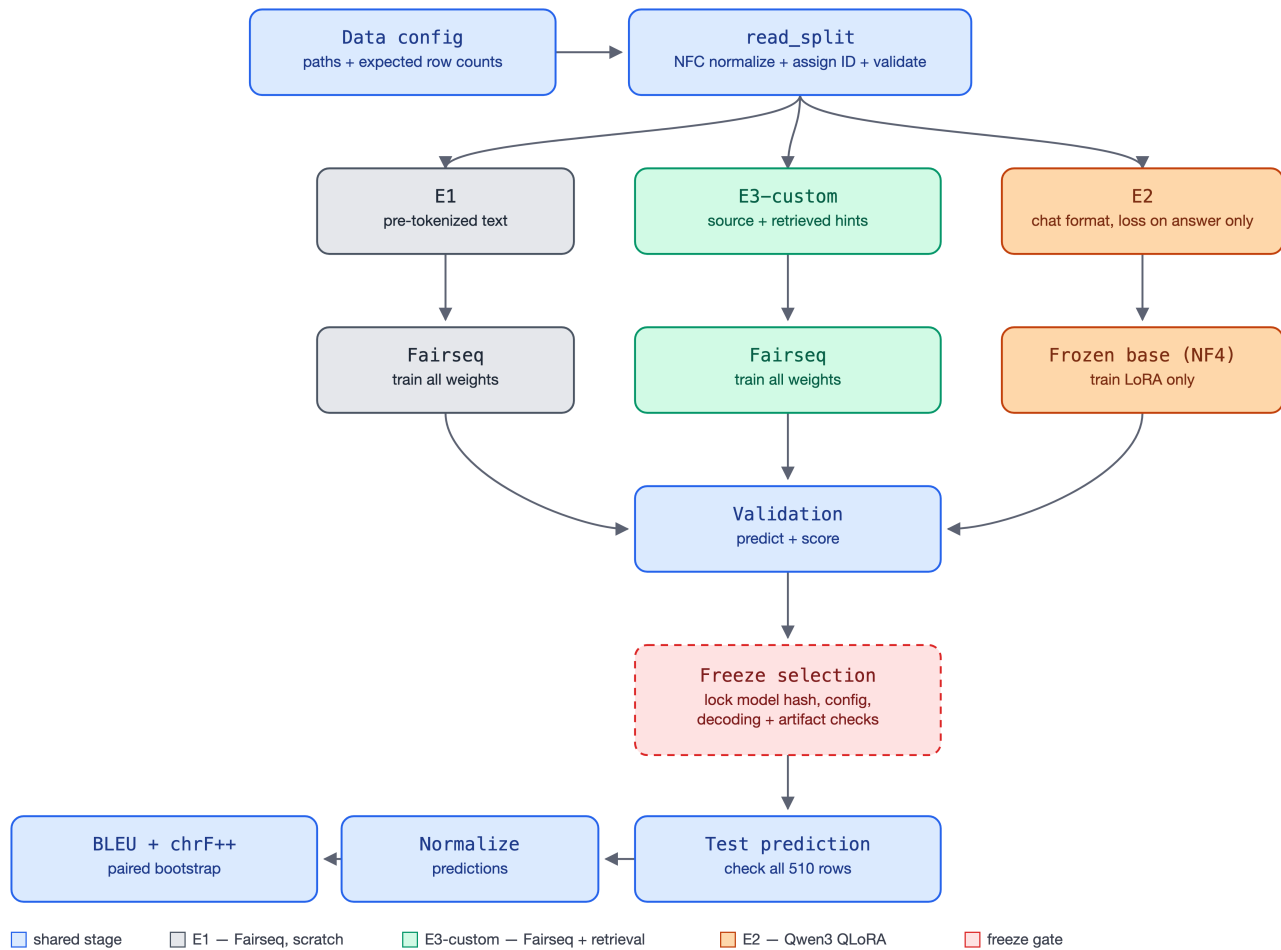
Ngữ liệu có các hạn chế chính sau:

- **Quy mô nhỏ và phạm vi hẹp:** chỉ có 19.218 cặp huấn luyện thuộc một tác phẩm sử học. Vì vậy, mô hình có thể học tốt văn phong và công thức biểu đạt của *Đại Việt sử ký toàn thư* nhưng chưa chắc tổng quát hóa sang thơ, văn bia, chiếu biểu hoặc các văn bản cổ thuộc giai đoạn khác.
- **Nhiều từ cần chỉnh tự động:** tập train là dữ liệu *silver*, nên có thể tồn tại cặp lệch câu, gộp–tách không tương ứng hoặc bản dịch chỉ tương đương một phần. Độ dài cực trị và tỷ lệ độ dài bất thường ở một số cặp là dấu hiệu cần được theo dõi, nhưng chưa đủ để tự động kết luận một cặp là sai.
- **Trùng lặp trong tập train:** khoảng 8,3% số dòng là lần lặp của các cặp đã xuất hiện. Điều này làm giảm lượng thông tin độc lập và có thể khiến mô hình thiên lệch về các cấu trúc biên niên có tần suất cao.
- **Phân bố ký tự đuôi dài:** 1.137 Hán tự trong tập train chỉ xuất hiện một lần. Các thực thể lịch sử và ký tự hiếm vì thế khó được học ổn định, đặc biệt đối với Transformer huấn luyện từ đầu mà không có tri thức tiền huấn luyện hoặc nguồn hỗ trợ.
- **Mất thông tin do chuẩn hóa:** việc bỏ dấu câu làm giảm tín hiệu về ranh giới mệnh đề và cấu trúc cú pháp; chuyển tiếng Việt về chữ thường làm mất dấu hiệu nhận biết tên riêng. Các phép biến đổi này giúp thống nhất dữ liệu nhưng có thể làm tăng độ mơ hồ của câu nguồn.
- **Thiếu ngữ cảnh tài liệu:** mỗi mẫu được xử lý như một cặp câu độc lập, trong khi việc giải thích đại từ tình lược, chức danh, niên hiệu và thực thể trong sử liệu thường cần các câu trước hoặc sau. Mô hình cấp câu vì thế có thể chọn bản dịch hợp lý cục bộ nhưng không nhất quán trong toàn đoạn.
- **Chỉ có một câu tham chiếu:** validation và test cung cấp một bản Hán văn cho mỗi câu tiếng Việt. Các cách diễn đạt khác vẫn đúng nghĩa có thể bị BLEU hoặc chrF++ đánh giá thấp; ngược lại, điểm toàn câu có thể che khuất lỗi nghiêm trọng ở một thực thể hoặc mốc thời gian.
- **Chưa loại trừ trùng lặp gần nghĩa:** kiểm tra hiện tại chỉ xác nhận không có giao nhau chính xác giữa các phân hoạch. Các câu gần trùng, biến thể chính tả hoặc những đoạn có nội dung tương tự vẫn có thể tồn tại và cần phương pháp phát hiện ngữ nghĩa sâu hơn nếu muốn kiểm toán rõ ràng dữ liệu toàn diện.
- **Hạn chế công bố:** ngữ liệu cốt lõi là tài sản nghiên cứu của phòng thí nghiệm và không được phân phối công khai [2]. Quy định này bảo vệ dữ liệu nhưng làm hạn chế khả năng kiểm chứng độc lập và tái lập hoàn toàn bởi các nhóm bên ngoài.

3 Phương pháp

3.1 Tổng quan pipeline

Hình 1 minh hoạ toàn bộ pipeline dùng chung cho ba thí nghiệm: hai đầu được chia sẻ, phần giữa là ba quy trình khác nhau. Việc nhìn tổng thể trước khi đi vào chi tiết từng thí nghiệm (mục 3.3–3.5) giúp thấy rõ đâu là phần chung, đâu là phần riêng biệt tạo nên khác biệt giữa các hệ thống.



Hình 1: Tổng quan pipeline

Ở đầu vào, cả ba thí nghiệm cùng đi qua một cấu hình dữ liệu và bước `read_split` duy nhất (mục 3.1): chuẩn hoá NFC, gộp khoảng trắng, gán định danh mẫu và kiểm tra toàn vẹn số dòng. Từ đây, pipeline rẽ thành ba nhánh độc lập. E1 giữ nguyên văn bản đã tách token sẵn và huấn luyện toàn bộ trọng số một Transformer Fairseq từ đầu. E3-custom đi theo đúng con đường Fairseq như E1 nhưng bổ sung thêm gợi ý Hán tự truy hồi vào câu nguồn trước khi huấn luyện —cũng huấn luyện toàn bộ trọng số, chỉ khác ở dữ liệu đầu vào. E2 tách khỏi con đường Fairseq hoàn toàn và chạy hoàn toàn dựa vào việc chúng ta thiết kế system prompt: dữ liệu được đóng gói thành định dạng hội thoại với loss chỉ tính trên phần trả lời, và chỉ một phần rất nhỏ tham số (adapter LoRA) được huấn luyện trên một mô hình nền đã đóng băng và lượng tử hoá 4-bit.

Ba nhánh hội tụ trở lại ở một quy trình đánh giá dùng chung. Cả ba mô hình đều được dùng để dự đoán và chấm điểm trên tập validation; kết quả này quyết định việc chốt một cấu hình mô hình duy nhất cho mỗi thí nghiệm —khóa lại hash mô hình, cấu hình, cách decode ra tiếng Hán để đảm bảo thí nghiệm có thể chạy lại với kết quả y hệt, và kiểm tra artifact trước khi chúng ta thực hiện dự đoán trên tập test. Dự đoán cuối cùng được chuẩn hoá về cùng một dạng chấm điểm (mục 3.6) trước khi tính BLEU và chrF++, và chênh lệch giữa các thí nghiệm được kiểm định bằng paired bootstrap (mục 3.7).

3.2 Dữ liệu và Tiền xử lý

Trước khi dữ liệu song ngữ Việt–Hán văn cổ được đưa vào bất kỳ thí nghiệm nào, toàn bộ ba phân hoạch (train, validation, test) đều đi qua một tầng tiền xử lý dùng chung, gồm ba thành phần: (i) một bước *chuẩn hoá văn bản* áp dụng đồng nhất cho cả hai phía nguồn/đích; (ii) một bước *đọc và kiểm tra toàn vẹn dữ liệu* chịu trách nhiệm gán định danh mẫu và chặn đứng mọi lệch lạc về số dòng; và (iii) một tầng *biểu diễn văn bản đích dưới nhiều dạng* phục vụ ba mục đích khác nhau: huấn luyện, hiển thị, và chấm điểm. Tầng tiền xử lý này được cài đặt trong `code/mt_pipeline/normalize.py` và `code/mt_pipeline/data.py`, và là điểm khởi đầu chung cho cả ba thí nghiệm E1, E2 và E3-custom trước khi mỗi thí nghiệm rẽ theo hướng chuẩn bị dữ liệu riêng của mình.

3.2.1 Chuẩn hoá văn bản

Hàm `normalize_whitespace` được áp dụng cho cả câu nguồn tiếng Việt lẫn câu đích Hán văn ngay khi đọc dữ liệu:

$$\text{normalize_whitespace}(s) = \text{strip}\left(\text{collapse}_{s+\rightarrow} \circ (\text{NFC}(s))\right), \quad (1)$$

trong đó s là chuỗi văn bản thô đọc từ tệp corpus, $\text{NFC}(\cdot)$ là phép chuẩn hoá Unicode theo dạng *Normalization Form Canonical Composition*, $\text{collapse}_{s+\rightarrow} \circ (\cdot)$ là phép thay thế mọi chuỗi khoảng trắng liên tiếp bằng đúng một dấu cách, và $\text{strip}(\cdot)$ là phép cắt bỏ khoảng trắng thừa ở hai đầu chuỗi.

Bước NFC là cần thiết vì tiếng Việt có dấu thanh và dấu phụ có thể được mã hoá theo hai cách tương đương nhưng khác biểu diễn byte: dạng *dựng sẵn* (một mã Unicode duy nhất cho cả âm tiết có dấu) và dạng *tổ hợp* (ký tự gốc cộng thêm một dấu kết hợp riêng biệt). Lấy ví dụ như: chữ "ă" có thể là một mã Unicode duy nhất U+1EA3 (dạng dựng sẵn), hoặc là hai mã Unicode nối tiếp nhau "a" (U+0061) + dấu hỏi kết hợp (U+0309) (dạng tổ hợp) —hai chuỗi byte hoàn toàn khác nhau nhưng hiển thị giống hệt "ă". Nếu không ép cả hai về cùng một dạng, hai câu nhìn giống hệt nhau có thể bị xem là khác nhau khi so khớp chuỗi, dẫn tới sai lệch khi gán định danh, kiểm tra trùng lặp hay tra cứu từ điển; sau khi qua $\text{NFC}(\cdot)$, cả hai đều quy về đúng một dạng U+1EA3. Tương tự, với một dòng thô đọc từ tệp có khoảng trắng không nhất quán, chẳng hạn "Vua sai\tnười" (hai dấu cách liên tiếp rồi một tab), phép $\text{collapse}_{s+\rightarrow} \circ (\cdot)$ gộp toàn bộ chuỗi khoảng trắng đó lại thành đúng một dấu cách, cho ra "Vua sai người" —đảm bảo ranh giới token luôn nhất quán bất kể tệp gốc chứa tab, xuống dòng thừa hay nhiều dấu cách liên tiếp do lỗi định dạng khi xuất corpus.

3.2.2 Đọc dữ liệu và kiểm tra toàn vẹn

Hàm `read_split` (`data.py`) là điểm đọc dữ liệu duy nhất của toàn bộ pipeline. Với mỗi phân hoạch, hàm này: đọc đồng thời hai tệp nguồn và đích; buộc dừng chương trình nếu số dòng hai phía không bằng nhau; buộc dừng chương trình nếu số dòng đọc được khác với số dòng kỳ vọng `expected_lines` khai báo trong cấu hình dữ liệu; gán cho mỗi cặp câu một định danh duy nhất theo quy tắc `sample_id = f"{split}-{index:06d}"`; và áp dụng phép chuẩn hoá ở Công thức 1 cho cả hai phía.

Lấy ví dụ như: với phân hoạch `train` và dòng thứ 42 (đếm từ 1), định danh mẫu được gán là `sample_id = "train-000042"` —định danh này giữ nguyên không đổi xuyên suốt pipeline, dùng để đối chiếu dự đoán với tham chiếu và để ghép cặp đúng câu khi so sánh hai hệ thống ở mục 3.7. Về hai điều kiện hard-fail: giả sử tệp nguồn tiếng Việt của tập `train` có 19.218 dòng nhưng tệp đích Hán văn chỉ có 19.217 dòng (một dòng bị rơi mất do lỗi khi xuất tệp, hoặc một bản sửa tệp không đồng bộ giữa hai phía) —`read_split` sẽ dừng chương trình ngay lập tức với lỗi báo số dòng hai phía không khớp, thay vì âm thầm ghép lệch câu nguồn với câu đích của dòng kế tiếp và để lỗi đó lan vào toàn bộ quá trình huấn luyện mà không ai phát hiện. Tương tự, nếu cấu hình khai báo `expected_lines: 19218` cho tập `train` nhưng tệp thực tế chỉ có 19.000 dòng (ví dụ do tải nhầm một phiên bản corpus cũ hoặc bị cắt xén khi sao chép), hàm cũng dừng ngay thay vì lặng lẽ huấn luyện trên một tập dữ liệu nhỏ hơn dự kiến. Hai điều kiện hard-fail này đóng vai trò một cổng toàn vẹn dữ liệu, đảm bảo một tệp corpus bị hoán đổi nhầm hoặc bị cắt xén không thể lọt vào một lượt chạy thực nghiệm mà không bị phát hiện ngay từ bước đọc dữ liệu đầu tiên.

3.2.3 Ba dạng biểu diễn của câu đầu ra

Vì phía Hán văn được cung cấp sẵn dưới dạng tách ký tự, mỗi dòng dữ liệu đích cần được biểu diễn dưới ba dạng khác nhau tùy mục đích sử dụng:

Bang 2: Ba dạng biểu diễn của câu đích Hán văn

Dạng	Định nghĩa	Dùng cho
<code>reference_tokenized</code>	dòng gốc đã chuẩn hoá khoảng trắng	mục tiêu huấn luyện/
<code>reference</code>	<code>detokenize_chinese</code> : nối ký tự, bỏ khoảng trắng	câu tham chiếu người
<code>reference_scored</code>	<code>score_form_chinese</code> : tách lại từng ký tự, nối bằng một dấu cách	đầu vào chấm điểm B

Về mặt hình thức:

```
reference = join(reference_tokenized)
```

còn

```
reference_scored = join», »(split_codepoint(reference))
```

—tức `reference_scored` là một phép tách-gộp-tách lại đi qua dạng không khoảng trắng.

Lấy ví dụ như: với `reference_tokenized` là " 如使越人", áp dụng `detokenize_chinese` cho ra `reference = " 如使越人"` (đã bỏ hết khoảng trắng). Từ `reference` này, áp dụng tiếp `score_form_chinese` —tách lại thành từng ký tự Unicode riêng lẻ rồi chèn một dấu cách giữa mỗi ký tự —cho ra `reference_scored = " 如使越人"`, **trùng khớp tuyệt đối** với `reference_tokenized` ban đầu. Kết quả này không phải ngẫu nhiên: nó minh hoạ chính xác ý nghĩa của phép round-trip —đi một vòng qua dạng không khoảng trắng rồi tách trở lại, ta thu được đúng dạng ban đầu, chứng tỏ

không mất thông tin gì trong quá trình. Bước tách–gộp–tách lại này được áp dụng như nhau cho mọi hệ thống, kể cả những hệ thống không tự nhiên sinh ra dạng tách-ký-tự (như E2, vốn sinh Hán văn liền mạch giống hệt dạng **reference**): nhờ vậy, dự đoán của cả ba thí nghiệm đều được ép về cùng một chuẩn (một Hán tự = một token) trước khi đưa vào chấm điểm, đảm bảo phép so sánh BLEU/chrF++ công bằng dù các hệ thống có định dạng đầu ra thô khác nhau.

Phép round-trip này chỉ không mất thông tin khi mọi token gốc trong corpus đúng là một codepoint duy nhất; điều kiện này đã được xác minh thực nghiệm trên toàn bộ 20.238 dòng của cả ba phân hoạch, không có ngoại lệ. Việc tách riêng ba dạng biểu diễn này là điều kiện cần để các chỉ số đánh giá (mục 3.6) được định nghĩa nhất quán giữa các thí nghiệm có định dạng đầu ra khác nhau.

3.2.4 Chính sách không dùng subword

Khi xây dựng từ điển cho Fairseq, ngưỡng tần suất tối thiểu được đặt bằng 1 ở cả hai phía (`--thresholdsrc 1 --thresholdtgt 1`), nghĩa là không áp dụng bất kỳ thuật toán ghép subword nào (không BPE, không WordPiece) và toàn bộ từ vựng xuất hiện trong tập train đều được giữ nguyên. Kết quả: từ điển tiếng Việt có 3.616 loại từ (word-level), từ điển Hán văn có 5.392 loại ký tự (character-level). Lựa chọn này có chủ đích: ghép các chuỗi Hán tự liên tiếp thành đơn vị subword theo thống kê đồng xuất hiện chỉ đáng tin cậy khi học trên một corpus đủ lớn; với một corpus nhỏ, phạm vi hẹp như *Đại Việt sử ký toàn thư*, một mô hình subword riêng có nguy cơ tạo ra các đơn vị ghép không ổn định, phụ thuộc mạnh vào các mẫu câu lặp lại đặc trưng của corpus hơn là phản ánh đúng cấu trúc ngôn ngữ. Cái giá phải trả: từ điển cố định không sinh được những ký tự chỉ xuất hiện ở tập giữ riêng —34 token đích ngoài từ điển ở validation, 45 ở test.

3.3 Thí nghiệm 1 (E1): Transformer huấn luyện từ đầu

3.3.1 Kiến trúc Transformer

Transformer [9] là một kiến trúc mạng nơ-ron cho bài toán chuỗi sang chuỗi, gồm hai khối *encoder* và *decoder*, xây dựng hoàn toàn từ cơ chế *self-attention* thay vì hồi quy (RNN/LSTM) hay tích chập (CNN). Thành phần cốt lõi là *scaled dot-product attention*:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \quad (2)$$

trong đó Q (query), K (key) và V (value) là ba phép chiếu tuyến tính khác nhau của cùng một biểu diễn đầu vào; tích $QK^\top$ đo độ tương đồng giữa mỗi cặp vị trí, chia cho $\sqrt{d_k}$ để tránh giá trị quá lớn làm softmax bão hoà; kết quả softmax là trọng số attention dùng để lấy tổ hợp trọng số của V . *Multi-head attention* chạy song song h head attention trên h phép chiếu khác nhau rồi nối kết quả lại, giúp mô hình chú ý đồng thời tới nhiều loại quan hệ khác nhau giữa các vị trí trong câu.

Kiến trúc này phù hợp với dịch máy vì hai lý do: self-attention cho phép mỗi vị trí chú ý trực tiếp tới mọi vị trí khác qua một bước duy nhất (giảm vanishing gradient với câu dài, hữu ích khi trật tự từ giữa tiếng Việt và Hán văn cổ có thể lệch xa nhau); và vì không có hồi quy tuần tự như dạng cấu trúc RNN hay LSTM, toàn bộ chuỗi input vào trong mô hình sẽ được xử lý song song khi huấn luyện.

3.3.2 Fairseq

Fairseq [5] là một toolkit huấn luyện mô hình chuỗi mã nguồn mở, xây dựng trên PyTorch. Fairseq cung cấp cài đặt tham chiếu cho Transformer cùng hạ tầng huấn luyện/giải mã chuẩn hoá (gom batch theo token, checkpoint, beam search, v.v...). E1 dùng Fairseq thay vì tự cài đặt lại, để bám sát một cài đặt chuẩn đã được kiểm chứng rộng rãi, tập trung nỗ lực vào lựa chọn dữ liệu/cấu hình thay vì hiện thực hoá mô hình.

3.3.3 Cấu hình và hàm mục tiêu

E1 dùng Transformer encoder-decoder tiêu chuẩn: sáu lớp mỗi khối, biểu diễn 512 chiều, feed-forward 2.048 chiều, tám attention head, dropout 0,1. Embedding đầu vào và ma trận chiếu đầu ra của decoder chia sẻ trọng số. Mô hình có 67.652.608 tham số, tối ưu từ đầu, không qua fine-tuning. Ở phía tiếng Việt, đơn vị đầu vào là token có sẵn trong corpus; ở phía Hán văn, mỗi ký tự là một token, theo đúng chính sách không dùng BPE đã nêu ở mục 3.1.

Với câu đích $y_{1:T}$, E1 tối thiểu hoá hàm cross-entropy có label smoothing $\varepsilon = 0,1$:

$$\mathcal{L}_{E1} = - \sum_{t=1}^T \left[(1 - \varepsilon) \log p(y_t \mid y_{<t}, x) + \frac{\varepsilon}{|V|} \sum_{v \in V} \log p(v \mid y_{<t}, x) \right]. \quad (3)$$

Bộ tối ưu Adam dùng $\beta = (0,9, 0,998)$, learning rate cực đại 5×10^{-4} và lịch `inverse_sqrt` sau 8.000 update warmup. Mỗi GPU nhận tối đa 1.000 token; `update_freq=4` tạo batch hiệu dụng khoảng 4.000 token. Huấn luyện chạy ở fp16 trên một GPU, giới hạn 50.000 update hoặc 500 epoch, patience 20 theo BLEU validation. Cần lưu ý checkpoint được báo cáo đã *warm-start*: nạp lại trọng số từ một lần chạy trước bị gián đoạn ở epoch 5 (update 579), sau đó tiếp tục và early-stop ở epoch 171 (24.791 update) —một hạn chế về khả năng tái lập từ một lần chạy sạch cần ghi nhận.

3.3.4 Giải mã bằng beam search

Thay vì luôn chọn token xác suất cao nhất ở mỗi bước (*greedy decoding*, dễ mắc kẹt vào chuỗi con tối ưu cục bộ), E1 dùng beam search với $B = 7$: duy trì đồng thời B giả thuyết có tổng log-xác suất cao nhất, mở rộng mỗi giả thuyết bằng mọi token kế tiếp, chỉ giữ lại B chuỗi tốt nhất cho bước sau, lặp tới khi mọi beam kết thúc câu hoặc chạm giới hạn độ dài.

Vì tổng log-xác suất giảm dần theo số token, một chuỗi ngắn hơn sẽ có điểm cao hơn một cách giả tạo nếu không hiệu chỉnh. Fairseq chuẩn hoá điểm theo độ dài với hệ số length penalty α :

$$\text{score}(y) = \frac{1}{|y|^\alpha} \sum_{t=1}^{|y|} \log p(y_t \mid y_{<t}, x), \quad (4)$$

E1 dùng $\alpha = 1,0$ —điểm cuối chính là log-xác suất trung bình mỗi token, không thiên vị thêm ngoài chuẩn hoá độ dài tự nhiên. Độ dài đích tối đa giới hạn theo $1,2|x| + 10$ token, với $|x|$ là độ dài câu nguồn —ngăn mô hình sinh chuỗi kéo dài vô hạn định với câu nguồn dài. Các giá trị beam, length penalty và giới hạn độ dài này được cố định trước khi chạy, không phải kết quả tìm kiếm siêu tham số.

3.3.5 Ví dụ minh hoạ toàn bộ quy trình

Để cụ thể hoá các khái niệm trên, xét một câu ví dụ minh hoạ (không phải câu thật trong corpus): câu nguồn tiếng Việt "Vua sai sứ sang nước Tống" cần dịch thành Hán văn cổ tương ứng "王遣使如宋".

Input đưa vào mô hình. Vì phía tiếng Việt đã được tách token sẵn trong corpus, câu nguồn là chuỗi 6 token ["Vua", "sai", "sứ", "sang", "nước", "Tống"]; câu đích Hán văn, tách theo ký tự, là chuỗi 5 token ["王", "遣", "使", "如", "宋"]. Sau bước tiền xử lý ở mục 3.1 (chuẩn hoá NFC + gộp khoảng trắng), hai chuỗi này được ghi ra file văn bản thô (text/train.vi, text/train.zh). Mô hình Transformer không đọc trực tiếp chuỗi ký tự —mỗi token được ánh xạ sang một chỉ số nguyên duy nhất trong từ điển tương ứng (dict.vi.txt cho phía nguồn, dict.zh.txt cho phía đích): ví dụ "Vua" → id 42, "sai" → id 803, chữ "王" → id 187, chữ "遣" → id 1204, v.v. Chuỗi chỉ số này (không phải chuỗi ký tự) mới là đầu vào số học thực sự được đưa vào embedding layer của encoder/decoder.

Fairseq construct những gì. Bước nhị phân hoá chạy đúng một lệnh:

```
fairseq-preprocess --source-lang vi --target-lang zh
--trainpref text/train --validpref text/valid --testpref text/test
--destdir data-bin --thresholdsrc 1 --thresholdtgt 1 --workers 4
```

Lệnh này quét toàn bộ text/train.{vi,zh}, xây hai từ điển dict.vi.txt/dict.zh.txt (mỗi dòng là một token kèm tần suất xuất hiện, sắp theo tần suất giảm dần), rồi mã hoá mọi câu của cả ba phân hoạch thành cặp tệp nhị phân .bin/.idx (mảng chỉ số nguyên nén, đọc nhanh hơn nhiều so với parse văn bản mỗi epoch). Kết quả nằm trong thư mục data-bin/, là đầu vào duy nhất cho bước huấn luyện tiếp theo —tức Fairseq không "học" trực tiếp trên file text, mà học trên biểu diễn nhị phân đã được lập chỉ mục này. Bước huấn luyện sau đó gọi fairseq-train trên data-bin/ với toàn bộ cờ kiến trúc/tối ưu đã liệt kê ở mục trên (--arch transformer --encoder-layers 6 ... --lr 5e-4 ...), ghi checkpoint định kỳ vào checkpoint_dir.

Output nhìn như thế nào. Khi giải mã (dùng fairseq-generate trên checkpoint tốt nhất), với mỗi câu, công cụ in ra 4 dòng gắn theo chỉ số câu, ví dụ với câu thứ 3 (định dạng S-/T-/H-/D- là quy ước gốc của Fairseq, phần văn bản Hán/Việt giữ nguyên font thường vì font monospace không có glyph chữ Hán):

```
S-3  Vua sai sứ sang nước Tống
T-3  王遣使如宋
H-3  -0,3821  王遣使如宋
D-3  -0,3821  王遣使如宋
```

trong đó S là câu nguồn, T là câu đích tham chiếu, H là giả thuyết (hypothesis) tốt nhất cùng log-xác suất trung bình mỗi token của nó (chính là Công thức 4), và D là bản detokenize của H (ở đây trùng nhau vì không có hậu xử lý detokenize riêng phía Fairseq). Pipeline chỉ đọc dòng H-, tách ra được chuỗi ký tự "王遣使如宋" —đây chính là reference_tokenized-dạng của dự đoán. Từ đó, hai phép biến đổi ở mục 3.1 được áp dụng: detokenize_chinese cho ra prediction = "王遣使如宋" (dạng người đọc), rồi score_form_chinese cho ra prediction_scored = "王遣使如宋" —dạng dùng để chấm BLEU/chrF++ ở mục 3.6.

Beam search decode nhìn như thế nào. Để dễ minh hoạ, xét beam size $B = 3$ (nhỏ hơn $B = 7$ thực tế dùng, chỉ để bảng dễ nhìn); các giá trị log-xác suất dưới đây là số minh hoạ, không phải số thật từ mô hình đã huấn luyện.

Bang 3: Minh hoạ hai bước đầu của beam search ($B = 3$)

Bước	3 giả thuyết được giữ lại	Log-xác suất cộng dồn
1	”王”	-0,10
	”帝”	-0,90
	”上”	-1,20
2	”王遣”	-0,15
	”王命”	-0,90
	”帝遣”	-1,20

Ở bước 1, mô hình cho ra phân phối xác suất trên toàn bộ từ điển đích cho token đầu tiên; ba token có log-xác suất cao nhất được giữ làm 3 giả thuyết ban đầu. Ở bước 2, cả ba giả thuyết đều được mở rộng bằng *mọi* token tiếp theo có thể (không chỉ 3 token trong bảng —bảng chỉ hiện kết quả sau khi lọc), tạo ra rất nhiều chuỗi ứng viên độ dài 2; trong số đó, chỉ 3 chuỗi có tổng log-xác suất cộng dồn cao nhất được giữ lại cho bước 3 —ở đây là ”王遣”, ”王命” và ”帝遣”, còn mọi mở rộng khác của ”上” và các mở rộng kém hơn của ”帝” bị loại bỏ hoàn toàn dù tại bước 1 ”帝” từng lọt vào top-3. Quá trình lặp lại như vậy cho đến khi cả 3 giả thuyết sinh token kết thúc câu hoặc chạm giới hạn $1,2|x| + 10$ token; giả thuyết có điểm đã chuẩn hoá độ dài (Công thức 4) cao nhất trong số các giả thuyết đã kết thúc được chọn làm câu dịch cuối cùng —chính là dòng H-3 ở trên.

3.4 Thí nghiệm 2 (E2): Qwen3-8B với QLoRA

3.4.1 Mô hình ngôn ngữ lớn và Qwen3-8B

Khác với E1 huấn luyện Transformer từ đầu, E2 tinh chỉnh một mô hình ngôn ngữ lớn đã tiền huấn luyện: Qwen3-8B [?], một mô hình *decoder-only* khoảng 8 tỷ tham số. Một LLM decoder-only mô hình hoá xác suất chuỗi đích y tự hồi quy, có điều kiện trên chuỗi nhắc x :

$$P(y \mid x) = \prod_{t=1}^T p(y_t \mid x, y_{<t}). \quad (5)$$

x là chuỗi nhắc (system instruction cùng câu nguồn tiếng Việt), $y_{1:T}$ là bản dịch Hán văn cổ sinh ra. Mỗi thừa số được tính bằng cách cho tiền tố đi qua các khối self-attention xếp chồng với *causal mask*: điểm attention từ vị trí t đến mọi vị trí $> t$ bị gán $-\infty$ trước softmax, nên vị trí t chỉ ”nhìn thấy” chính nó và các vị trí trước.

Lý do chọn tinh chỉnh LLM có sẵn thay vì huấn luyện từ đầu như E1: với E1, mọi tri thức ngôn ngữ và dịch thuật phải học hoàn toàn từ 19.218 cặp câu. Qwen3-8B đã tiền huấn luyện trên kho ngữ liệu khổng lồ, nhiều khả năng gồm cả tiếng Việt, Hán văn hiện đại và một phần Hán văn cổ, nên đã có sẵn hiểu biết đáng kể về từ vựng/ngữ pháp/văn phong hai ngôn ngữ trước khi tiếp xúc dữ liệu đề tài. So sánh E1 với E2 là phép kiểm định cho giả thuyết: tri thức tiền huấn luyện kết hợp phương pháp tinh chỉnh tiết kiệm tham số có bù đắp được quy mô ngữ liệu song ngữ nhỏ hay không.

3.4.2 LoRA và QLoRA

Tinh chỉnh toàn bộ 8 tỷ tham số trên chỉ 19.218 câu vừa tốn bộ nhớ vừa dễ overfitting. E2 dùng LoRA [4]: giữ nguyên (đóng băng) ma trận trọng số $W \in \mathbb{R}^{d \times k}$, chỉ học cập nhật hạng thấp $\Delta W = BA$, với $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, $r \ll \min(d, k)$:

$$h = Wx + \frac{\alpha}{r} BAx. \quad (6)$$

r (rank) là chiều "cổ chai" của cập nhật, quyết định số tham số mới ($r(d+k)$ thay vì $d \times k$); đề tài dùng $r = 16$. α là hệ số tỷ lệ cố định tách độ lớn cập nhật khỏi giá trị r ; dùng $\alpha = 32$, cho tỷ lệ $\alpha/r = 2$. **target_modules=all-linear** nghĩa là mọi lớp tuyến tính trong các khối Transformer đều được gán một cặp (A, B) riêng. Tại khởi tạo $B = 0$ nên $\Delta W = 0$, mô hình thích ứng ban đầu giống hệt mô hình nền; chỉ A, B được huấn luyện.

QLoRA [3] bổ sung lượng tử hoá phần trọng số nền đã đóng băng: **NF4** (4-bit NormalFloat) đặt 16 mức lượng tử tại các phân vị của phân phối chuẩn thay vì chia đều, tối ưu cho trọng số có phân phối gần chuẩn; **double quantization** lượng tử hoá thêm chính các hằng số tỷ lệ; **compute dtype fp16** giải lượng tử tức thời chỉ để nhân ma trận; **paged optimizer (paged_adamw_8bit)** dùng phân trang bộ nhớ hợp nhất để tránh hết bộ nhớ. Kết quả: khoảng 8 tỷ tham số nền chỉ chiếm khoảng một phần tư dung lượng so với fp16, trong khi chỉ các ma trận LoRA —một phần rất nhỏ— thực sự được huấn luyện.

3.4.3 Biểu diễn dữ liệu và hàm mất mát

Khác E1/E3-custom dùng **reference_tokenized** làm đích, E2 dùng **reference** (đã detokenize) —mô hình học sinh Hán văn tự nhiên, không phải chuỗi cách nhau bởi khoảng trắng. Mỗi mẫu được dựng thành hội thoại hai lượt (system instruction + câu nguồn), mã hoá hai lần qua **apply_chat_template** —một lần chỉ tiền tố, một lần gồm cả câu trả lời—và khẳng định chuỗi đầy đủ bắt đầu đúng bằng chuỗi tiền tố (kiểm tra an toàn chống lệch mask). Nhãn huấn luyện:

$$\text{label}_i = \begin{cases} -100, & i < |P| \\ \text{token}_i, & i \geq |P| \end{cases} \quad (7)$$

với P là chuỗi token tiền tố. Giá trị -100 khiến cross-entropy bỏ qua vị trí đó —loss chỉ tính trên token bản dịch. Nhiều mẫu hoàn chỉnh được đóng gói (packing) thành cửa sổ 768 token sau xáo trộn có seed; 19.218 mẫu đóng gói thành 2.903 chuỗi huấn luyện.

3.4.4 Huấn luyện

Tối ưu **paged_adamw_8bit**, lr 2×10^{-4} , lịch cosine, warmup 0,05, weight decay 0,01, grad-norm clip 1,0. Batch hiệu dụng: 1 mẫu/thiết bị $\times$ 16 bước tích lũy gradient, 3 epoch (546 bước), fp16. Checkpoint triển khai là checkpoint có **eval_loss** thấp nhất, ở bước 400 (khoảng epoch 2,2/3) —không phải cuối huấn luyện—được khôi phục (**load_best_model_at_end**) trước khi lưu.

3.4.5 Giải mã

Beam search với beam = 4, tắt định (**do_sample=false**): luôn chọn tiếp tục có xác suất tích lũy cao nhất, không sampling. Số token tối đa thích ứng theo độ dài nguồn: $\min(512, \lceil 2 \times L_{\text{src}} \rceil + 32)$. Chuỗi nhắc khi giải mã dựng lại giống hệt lúc huấn luyện, đảm bảo không lệch hình dạng giữa hai giai đoạn.

3.4.6 Ví dụ minh hoạ toàn bộ quy trình

Input và chuỗi nhắc. Với cùng câu ví dụ "Vua sai sứ sang nước Tống" (dịch: "王遣使如宋", đã detokenize, không khoảng trắng), E2 dựng hội thoại hai lượt:

system: Bạn là hệ thống dịch văn bản lịch sử. Hãy dịch câu tiếng Việt sang Hán văn cổ. Chỉ trả về bản dịch Hán văn, không giải thích.

user: Dịch sang Hán văn cổ:

Vua sai sứ sang nước Tống

Chuỗi này được mã hoá hai lần: *tiền tố* (chỉ system+user, cộng thêm token báo hiệu lượt trả lời sắp tới) và *đầy đủ* (tiền tố cộng thêm câu trả lời "王遣使如宋"). Giả sử (số minh hoạ) tiền tố mã hoá thành $|P| = 38$ token, và chuỗi đầy đủ mã hoá thành 43 token (5 token còn lại ứng với 5 ký tự bản dịch). Mã nguồn khẳng định `full_ids[:38] == prefix_ids` —nếu không đúng (ví dụ do thay đổi chat template làm lệch điểm cắt), quá trình huấn luyện dừng ngay thay vì âm thầm tính loss sai vị trí.

Nhãn huấn luyện (loss mask). Áp dụng Công thức 7 với $|P| = 38$: 38 vị trí đầu (toàn bộ system + user) nhận nhãn -100 , chỉ 5 vị trí cuối (ứng với 5 ký tự "王", "遣", "使", "如", "宋") mang token ID thật để tính loss:

`labels = [-100, -100, ..., -100, (38 lần), id("王"), id("遣"), id("使"), id("如"), id("宋")]`

Mô hình không bao giờ bị phạt vì "đoán sai" phần chỉ dẫn hệ thống hay câu nguồn —điều đó vốn đã cho sẵn, chỉ có 5 token cuối mới đóng góp vào gradient.

LoRA vận hành cụ thể ra sao. Xét ví dụ số học thu nhỏ (không phải kích thước thật của Qwen3-8B) để thấy Công thức 6 tính toán thế nào: giả sử một ma trận trọng số $W \in \mathbb{R}^{4 \times 4}$ trong một lớp tuyến tính, hạng $r = 2$ (thay vì 16 thật), và tỷ lệ $\alpha/r = 2$ (giữ đúng tỷ lệ $32/16 = 2$ của cấu hình thật):

$$W = \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{pmatrix}, \quad A = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 0 & 0 \end{pmatrix}.$$

Với đầu vào $x = (1, 1, 0, 0)^\top$: $Wx = (1, 1, 2, 0)^\top$ (phần đóng góp từ mô hình nền, không đổi khi huấn luyện). Thay vì nhân trực tiếp BA (ra một ma trận 4×4 đầy đủ), LoRA tính qua hai bước rẻ hơn nhiều: $Ax = (1, 1)^\top$ (chỉ 2 chiều), rồi $B(Ax) = (1, 1, 2, 0)^\top$. Nhân với tỷ lệ $\alpha/r = 2$: $\frac{\alpha}{r}B(Ax) = (2, 2, 4, 0)^\top$. Kết quả cuối:

$$h = Wx + \frac{\alpha}{r}BAx = (1, 1, 2, 0)^\top + (2, 2, 4, 0)^\top = (3, 3, 6, 0)^\top.$$

Điểm mấu chốt về hiệu quả bộ nhớ/tính toán: LoRA không bao giờ cần dựng ma trận 4×4 đầy đủ của BA , chỉ cần hai phép nhân nhỏ qua chiều r . Ở quy mô thật ($d = k = 4096$ chẳng hạn, một bậc kích thước hợp lý cho các lớp tuyến tính của một mô hình 8 tỷ tham số), một ma trận W đầy đủ có $4096 \times 4096 \approx 16,8$ triệu tham số; với $r = 16$, cặp A, B chỉ cần $16 \times (4096 + 4096) \approx 131$ nghìn tham số —ít hơn khoảng 128 lần. Vì `target_modules=all-linear` áp dụng điều này cho *mọi* lớp tuyến tính trong mạng, mức tiết kiệm này nhân lên trên toàn bộ 8 tỷ tham số nền, giải thích vì sao chỉ một phần rất nhỏ thực sự cần huấn luyện và lưu trữ ở độ chính xác cao.

Giải mã. Với câu nguồn "Vua sai sứ sang nước Tống" giả sử mã hoá thành $L_{\text{src}} = 9$ token (theo tokenizer của Qwen3), giới hạn sinh token thích ứng là $\min(512, \lceil 2 \times 9 \rceil + 32) = \min(512, 50) = 50$ token —đủ rộng cho một bản dịch 5 ký tự nhưng không cho phép mô hình sinh tràn lan nếu lặp vòng. Vì `do_sample=false`, quá trình beam search ở đây hoàn toàn tắt định: với cùng một checkpoint và cùng một câu nguồn, chạy lại nhiều lần luôn cho ra đúng một kết quả —khác với sinh văn bản có *sampling* (nơi mỗi lần chạy có thể ra kết quả khác nhau do yếu tố ngẫu nhiên trong việc chọn token).

3.5 Thí nghiệm 3 (E3-custom): Fairseq với truy hồi tri thức nguồn

Về kiến trúc, hàm mục tiêu, bộ tối ưu và giải mã, E3-custom hoàn toàn giống E1 (mục 3.3). Khác biệt duy nhất là một can thiệp *chỉ ở tiền xử lý*: trước khi mã hóa nhị phân, câu nguồn được bổ sung một danh sách Hán tự gợi ý truy hồi từ một từ điển tri thức có cấu trúc.

Cần định vị đúng kỹ thuật này: đây không phải RAG truy hồi đoạn văn bằng embedding, cũng không phải fusion bộ nhớ dịch thuật truy hồi nguyên cặp câu. Đây là *truy hồi token ứng viên dựa trên từ điển* (lexicon-based candidate-token retrieval) / *chèn gợi ý phía nguồn*: truy hồi một danh sách ngắn ký hiệu đơn lẻ (từng Hán tự) bằng một luật xếp hạng hoàn toàn xác định, không có embedding hay điểm tương đồng học được —một can thiệp thuần dữ liệu, không đổi kiến trúc.

3.5.1 Xây dựng chỉ mục tri thức

`knowledge.json` là từ điển có cấu trúc cho khoảng 16.000 mục Hán/Nôm tự. Với mỗi mục, các chuỗi truy vấn tiếng Việt được thu thập từ `pronunciation`, `nom_query`, `lookup_query`, cùng `query_variants`; mỗi chuỗi chuẩn hoá khoảng trắng, casefold, tách thành bộ token làm khóa cho một chỉ mục bộ-token $\rightarrow$ tập Hán tự. Kết quả: **6.389** dạng truy vấn duy nhất. Song song, một bảng tần suất đếm mọi Hán tự đích xuất hiện *chỉ trong tập train* —bắt buộc giới hạn ở train để tránh rò rỉ thông tin đánh giá ngược vào truy hồi.

3.5.2 Quy tắc xếp hạng

Thuật toán trượt cửa sổ qua mọi cặp (vị trí, độ dài dạng truy vấn) trong câu nguồn. Mỗi lần một đoạn token khớp chính xác một dạng truy vấn đã lập chỉ mục (một *hit*), mọi Hán tự thuộc tập ứng với dạng đó được cộng 1 điểm. Các ứng viên được xếp hạng theo:

$$\text{rank}(c) = (-\text{match_count}(c), -\text{train_frequency}(c), c). \quad (8)$$

Sắp xếp tăng dần theo bộ ba này tương đương ba tầng ưu tiên: (1) số lần khớp giảm dần; (2) khi bằng nhau, tần suất trong tập train giảm dần; (3) khi vẫn bằng nhau, mã Unicode của ký tự tăng dần —một tiêu chí phân định cuối cùng đảm bảo toàn quy trình xác định tuyệt đối, không ngẫu nhiên.

3.5.3 Kết xuất vào chuỗi nguồn

Các ứng viên đã xếp hạng được nối vào câu nguồn, phân cách bằng sentinel `__knowledge__`: câu nguồn cuối = câu gốc + `__knowledge__` + K ứng viên hàng đầu, với $K = \min(64, \text{ngân sách vị trí encoder còn đảm bảo không vượt quá } \text{max_source_positions}=256)$. Gần như mọi câu bão hòa ở trần 64: 100% câu ở cả ba tập được bổ sung gợi ý, trung bình 62 ứng viên/câu từ trung bình 20–21 đoạn khớp —độ phủ cao nhưng độ chọn lọc thấp.

3.5.4 Hệ quả đo được và hạn chế

Việc chèn gợi ý kéo theo hệ quả đo được cụ thể: token nguồn tập train tăng từ 421.215 lên **1.622.823**; từ vựng nguồn tăng từ 3.616 lên **17.264** loại; tham số mô hình tăng từ 67.652.608 lên **74.640.384** (+10,3%, chỉ ở embedding encoder vì `share_decoder_input_output_embed` chỉ chia sẻ trọng số decoder).

Trên test, E3-custom đạt BLEU 29,62/chrF++ 31,41 so với 28,63/30,58 của E1 — Δ BLEU = +0,99, [−0,14, 2,16], $p = 0,092$, không có ý nghĩa thống kê ở $\alpha = 0,05$ (thử tục đầy đủ ở mục 3.7). Quan trọng hơn, so sánh E1 với E3-custom **không kiểm soát** chỉ một biến, vì ba lý do đo được: (a) nhiều hơn 10,3% tham số; (b) chuỗi nguồn dài hơn 3,9 lần; và (c) —sự thật ở cấp lần chạy, không phải khác cấu hình —huấn luyện E3-custom bị `max_update=50.000` cắt ngang ở epoch 87, chỉ 10 epoch sau điểm tốt nhất (epoch 77, BLEU validation 31,05), khiến `patience=20` chưa kịp kích hoạt như ở E1 (chạy trọn 171 epoch tới hội tụ). Mức tăng +0,99 BLEU nên đọc là *chưa đo được sạch sẽ*, không phải bằng chứng truy hồi không giúp ích.

3.5.5 Ví dụ minh hoạ toàn bộ quy trình

Chỉ mục tri thức (rút gọn để minh hoạ). Giả sử `knowledge.json` có các mục sau liên quan tới câu ví dụ (số liệu minh hoạ, không phải nội dung thật của tập):

"vua" → {王} "sai" → {使} "sứ" → {使} "tổng" → {宋} "nước tổng" → {宋}

Ba dạng truy vấn đầu là dạng một token, hai dạng cuối là "tổng" (một token) và "nước tổng" (hai token) —cùng trở tới ký tự 宋, minh hoạ việc một ký tự có thể được truy hồi qua nhiều dạng truy vấn khác nhau, dài ngắn khác nhau.

Xếp hạng ứng viên cho câu ví dụ. Với câu nguồn đã casefold "vua sai sứ sang nước tổng" (6 token, đánh số vị trí 0–5), thuật toán trượt cửa sổ qua mọi (vị trí, độ dài) có trong chỉ mục (ở đây độ dài 1 và 2):

Bang 4: Các đoạn khớp (hit) khi trượt cửa sổ qua câu ví dụ

Vị trí	Đoạn khớp	Ký tự được cộng điểm
0	"vua"	王 (+1)
1	"sai"	使 (+1)
2	"sứ"	使 (+1)
4–5	"nước tổng"	宋 (+1)
5	"tổng"	宋 (+1)

Vị trí 3 ("sang") không khớp dạng truy vấn nào nên bị bỏ qua. Tổng điểm tích lũy: `match_count()` = 2 (từ "sai" và "sứ"), `match_count()` = 2 (từ "tổng" và "nước tổng"), `match_count()` = 1 (từ "vua") —tổng cộng `matched_spans` = 5.

Áp dụng Công thức 8: 使 và 宋 hoà nhau ở tầng (1) (cùng `match_count`=2), nên xét tới tầng (2) —tần suất trong tập train. Giả sử (số minh hoạ) 宋 xuất hiện 512 lần trong tập train, còn 使 xuất hiện 340 lần: 宋 có tần suất cao hơn nên xếp trước 使 dù `match_count` bằng nhau. 王 xếp cuối vì `match_count` thấp hơn (1). Kết quả xếp hạng cuối cùng: [, ,].

Kết xuất vào chuỗi nguồn. Với ngân sách còn đủ chỗ (giả sử cả 3 ứng viên đều nằm trong 64 slot cho phép), câu nguồn được kết xuất thành:

vua sai sứ sang nước tống __knowledge__ 宋使王

Đây chính là chuỗi được ghi vào `text/train.vi` rồi đưa qua `fairseq-preprocess` (mục 3.3) — từ góc nhìn của Fairseq, ba ký tự Hán 宋使王 chỉ là các token phụ nối thêm vào cuối câu nguồn, không có gì đặc biệt hơn phần câu nguồn gốc.

Nhận xét về độ phủ. Đáng chú ý: trong 3 ký tự được gợi ý (宋, 使, 王), cả 3 đều thực sự xuất hiện trong bản dịch đúng ”王遣使如宋” —nghĩa là ở ví dụ này, retrieval tìm đúng 3/5 ký tự đích (thiếu 遣 và 如, hai ký tự không có dạng truy vấn nào được lập chỉ mục khớp với câu nguồn). Đây là minh họa trực quan cho tính chất ”độ phủ cao nhưng không toàn phần” đã nêu ở phần ”Hệ quả đo được và hạn chế”: mô hình được cho một shortlist có ích nhưng không đầy đủ, vẫn phải tự suy ra phần còn lại (như 遣, 如) từ chính khả năng sinh ngôn ngữ của nó, không phải từ gợi ý truy hồi.

3.6 Chỉ số đánh giá

Chất lượng bản dịch đo bằng hai chỉ số cấp corpus bổ trợ nhau: BLEU [6] và chrF++ [7, 8], tính bằng `sacrebleu` với signature cố định để tái lập chính xác.

3.6.1 Chuẩn hoá trước khi chấm điểm

Trước khi chấm điểm, câu dự đoán và tham chiếu đi qua `score_form_chinese`: ghép thành văn bản liền mạch, tách lại từng ký tự Unicode, nối bằng một khoảng trắng —ép mọi đầu vào về cùng dạng chuẩn (một Hán tự = một token). Cần thiết vì E1/ E3-custom sinh Hán văn tách sẵn theo ký tự còn E2 sinh Hán văn tự nhiên không khoảng trắng; phép biến đổi này đã được kiểm chứng không mất thông tin trên toàn bộ 20.238 dòng.

3.6.2 BLEU

$$p_n = \frac{\sum_{\text{n-gram}} \min(\text{Count}_{\text{hyp}}(\text{n-gram}), \text{Count}_{\text{ref}}(\text{n-gram}))}{\sum_{\text{n-gram}} \text{Count}_{\text{hyp}}(\text{n-gram})}. \quad (9)$$

Số lần khớp bị cắt trần ở số lần xuất hiện nhiều nhất trong tham chiếu, tránh lặp một ký tự đúng nhiều lần để đẩy precision giả tạo.

$$BP = \begin{cases} 1 & \text{nếu } c > r \\ \exp\left(1 - \frac{r}{c}\right) & \text{nếu } c \leq r \end{cases} \quad (10)$$

với c , r là tổng độ dài dự đoán/tham chiếu toàn corpus.

$$\text{BLEU} = BP \cdot \exp\left(\sum_{n=1}^4 w_n \log p_n\right), \quad w_n = \frac{1}{4}. \quad (11)$$

Cấu hình: `lowercase=False`, `tokenize="13a"`, `smooth_method="exp"` (làm mượt khi $p_n = 0$ để tránh $\log p_n \rightarrow -\infty$), `effective_order=False`.

Lưu ý: vì đầu vào đã tách sẵn một ký tự một token trước khi vào `sacrebleu`, `tokenize="13a"` trở thành no-op —BLEU ở đây thực chất đo n-gram **cấp ký tự**, không phải cấp từ như cách hiểu thông thường; không nên so sánh trực tiếp với BLEU cấp từ của các cặp ngôn ngữ khác.

3.6.3 chrF++

$$chrP = \frac{\text{n-gram ký tự khớp}}{\text{n-gram ký tự trong dự đoán}}, \quad chrR = \frac{\text{n-gram ký tự khớp}}{\text{n-gram ký tự trong tham chiếu}}, \quad (12)$$

$$chrF_{\beta} = (1 + \beta^2) \cdot \frac{chrP \cdot chrR}{\beta^2 \cdot chrP + chrR}. \quad (13)$$

$\beta = 2$: recall được coi trọng gấp đôi precision. `char_order=6`: n-gram ký tự bậc 1–6. `word_order=2` là phần mở rộng chrF++ (n-gram cấp từ bậc 1–2), nhưng **vì văn bản đã tách sẵn một ký tự một token**, cơ chế tách "từ" theo khoảng trắng của `sacrebleu` coi mỗi ký tự là một "từ" — `word_order=2` ở đây thực chất tính lại bigram **cấp ký tự** một lần nữa, không phải bigram cấp từ thật. `whitespace=False`, `lowercase=False`.

3.6.4 Diễn giải thang điểm cho các chỉ số

Một thang diễn giải BLEU thường được trích dẫn trong cộng đồng dịch máy (ví dụ, tài liệu của Google Cloud Translation) là:

Bang 5: Thang diễn giải BLEU phổ biến (tính trên n-gram **cấp từ**, cặp ngôn ngữ giàu tài nguyên)

Điểm BLEU	Chất lượng
< 10	Gần như vô dụng
10–19	Khó nắm được ý chính
20–29	Ý chính rõ nhưng còn lỗi ngữ pháp đáng kể
30–40	Có thể hiểu được đến khá tốt
40–50	Chất lượng cao
50–60	Chất lượng rất cao, trôi chảy
> 60	Thường được đánh giá tốt hơn cả bản dịch của con người

Cảnh báo quan trọng —không áp trực tiếp thang này vào bài: thang trên được xây dựng trên BLEU tính n-gram **cấp từ**, thường trên các cặp ngôn ngữ giàu tài nguyên với văn bản đã tách từ theo nghĩa thông thường (ví dụ Anh–Pháp). Như đã nêu ở phần BLEU phía trên, điểm số trong đề tài này là BLEU **cấp ký tự**. Về bản chất thống kê, so khớp n-gram cấp ký tự dễ "trúng ngẫu nhiên" hơn nhiều so với so khớp n-gram cấp từ —một ký tự Hán đơn lẻ trùng nhau có xác suất xảy ra ngẫu nhiên cao hơn hẳn một từ trọn vẹn trùng nhau —nên BLEU cấp ký tự thường cho điểm số cao hơn một cách hệ thống so với BLEU cấp từ ở cùng một mức chất lượng dịch thực tế. Vì vậy, điểm BLEU 28–41 của các hệ thống trong đề tài **không thể** tra thẳng vào bảng trên để suy ra "chất lượng tương đương 28–41 điểm BLEU cấp từ" —làm vậy sẽ đánh giá sai lệch (có thể lạc quan quá mức) chất lượng dịch thực tế.

chrF++ cũng không có một thang diễn giải tuyệt đối được thống nhất rộng rãi như BLEU; thực hành phổ biến chỉ xem chrF trong khoảng 40–60 là "chấp nhận được" ở mức tổng quát cho nhiều cặp ngôn ngữ, nhưng con số này cũng được đo ở mức từ/ký tự hỗn hợp thông thường, không phải chế độ thuần ký tự như ở đây.

Cách diễn giải đúng cho đề tài này: vì không có một điểm neo tuyệt đối đáng tin cậy để so sánh chéo giữa các chế độ tính khác nhau, cách diễn giải có ý nghĩa nhất là so sánh *tương đối* giữa ba hệ thống trên *cùng* một chế độ tính (đúng những gì mục 3.7 làm): E2 đạt BLEU/chrF++ cao hơn rõ rệt và có ý nghĩa thống kê so với cả E1 lẫn E3-custom (Δ dương, khoảng tin cậy không chứa

0, $p = 0,002$); còn chênh lệch giữa E3-custom và E1 tuy điểm tuyệt đối cao hơn nhưng không đủ ý nghĩa thống kê. Nói cách khác, thứ tự xếp hạng giữa ba hệ thống đáng tin hơn nhiều so với việc gán nhãn "tốt" hay "xấu" cho một con số BLEU/chrF++ đơn lẻ tách khỏi ngữ cảnh so sánh nội bộ của đề tài.

3.7 Kiểm định ý nghĩa thống kê

Một điểm BLEU/chrF++ trên toàn tập test chỉ là một ước lượng điểm; vì đây là đại lượng tổng hợp phi tuyến trên corpus, độ biến thiên lấy mẫu không có công thức giải tích đóng. Quan sát "hệ thống B cao hơn A 0,99 điểm" chưa đủ để khẳng định đó là khác biệt thật hay chỉ ngẫu nhiên do 510 câu cụ thể của tập test này.

Bootstrap và vì sao phải bắt cặp

Từ tập test $n = 510$ câu, lặp B lần lấy mẫu ngẫu nhiên có hoàn lại để tạo B tập test giả lập, tính lại metric trên mỗi tập. Đề tài dùng *paired bootstrap* [?]: mỗi lần lặp b chỉ rút một bộ chỉ số câu, áp dụng **cùng** bộ đó cho cả hai hệ thống:

$$\Delta_b = \text{metric}(\text{candidate}_b) - \text{metric}(\text{baseline}_b), \quad b = 1, \dots, B. \quad (14)$$

Nếu resample độc lập, mỗi Δ_b trộn lẫn khác biệt chất lượng thật sự và nhiễu do câu dễ/khó rơi lệch giữa hai tập resample. Dùng chung một bộ chỉ số mỗi lần lặp khiến nhiễu do độ khó câu triệt tiêu, chỉ còn lại chênh lệch hệ thống mang tính hệ thống.

Lấy ví dụ như: để dễ hình dung, giả sử tập test chỉ có $n = 5$ câu (đánh số 1–5) thay vì 510 câu thật. Ở lần lặp $b = 1$, bộ chỉ số được rút ngẫu nhiên có hoàn lại là $[2, 2, 5, 1, 3]$ — chú ý câu số 2 xuất hiện hai lần, còn câu số 4 không được chọn lần nào, đúng bản chất "có hoàn lại" của bootstrap. Cả hai hệ thống baseline và candidate đều dùng *đúng* bộ chỉ số $[2, 2, 5, 1, 3]$ này để dựng lại corpus giả lập của mình (lấy đúng các dòng dự đoán số 2, 2, 5, 1, 3 theo thứ tự đó từ mỗi hệ thống) — đây chính là điểm "paired": nếu baseline dùng $[2, 2, 5, 1, 3]$ mà candidate lại dùng một bộ chỉ số khác độc lập, phần chênh lệch đo được sẽ lẫn cả nhiễu do hai hệ thống không được so trên cùng tập câu. Tính lại BLEU trên corpus giả lập 5 câu này cho mỗi hệ thống, giả sử ra $\text{baseline}_1 = 26,8$ và $\text{candidate}_1 = 29,5$, ta được $\Delta_1 = +2,7$. Lặp lại toàn bộ quy trình này $B = 1.000$ lần (với $n = 510$ câu thật) cho ra 1.000 giá trị $\Delta_1, \dots, \Delta_{1.000}$.

Khoảng tin cậy và p-value

$B = 1.000$, seed cố định 42. Khoảng tin cậy 95% là khoảng phân vị $[2, 5\%, 97, 5\%]$ của $\{\Delta_b\}$:

$$p = \min\left(1, 2 \cdot \min\left(\frac{\#\{b : \Delta_b \leq 0\} + 1}{B + 1}, \frac{\#\{b : \Delta_b \geq 0\} + 1}{B + 1}\right)\right). \quad (15)$$

Số hạng "+1" là làm tròn Laplace, đảm bảo p không bao giờ báo cáo chính xác bằng 0 dù B hữu hạn.

Lấy ví dụ như: sau khi có đủ 1.000 giá trị Δ_b , sắp xếp chúng tăng dần. Cận dưới của khoảng tin cậy 95% là phần tử thứ $\lceil 0,025 \times 999 \rceil + 1 = 25$ trong dãy đã sắp xếp, cận trên là phần tử thứ $\lceil 0,975 \times 999 \rceil + 1 = 975$ — chính là công thức tính khoảng phân vị đã dùng để ra các khoảng tin cậy ở bảng kết quả bên dưới. Về p-value: với so sánh E3-custom – E1, giả sử trong 1.000 lần resample có đúng 45 lần $\Delta_b \leq 0$ (tức phần lớn các lần resample cho thấy E3-custom vẫn nhỉnh hơn E1, nhưng

không phải luôn luôn —một số lần resample cho kết quả ngược lại) và do đó 955 lần $\Delta_b \geq 0$. Áp dụng công thức 15:

$$p = \min\left(1, 2 \cdot \min\left(\frac{45 + 1}{1.000 + 1}, \frac{955 + 1}{1.000 + 1}\right)\right) = \min(1, 2 \times 0,0459) \approx 0,092,$$

khớp đúng với giá trị $p = 0,092$ đã báo cáo cho ΔBLEU của E3-custom – E1 ở bảng dưới đây — minh hoạ rằng con số 0,092 không phải lấy từ đâu ra mà chính là kết quả của đúng phép đếm tỷ lệ đuôi này trên 1.000 lần resample thực tế.

Kết quả trên tập test

- **E2 – E1:** $\Delta\text{BLEU} = +12,14$ [10, 22, 13, 77], $p = 0,002$; $\Delta\text{chrF++} = +10,29$ [9, 03, 11, 79], $p = 0,002$.
- **E3-custom – E1:** $\Delta\text{BLEU} = +0,99$ [–0, 14, 2, 16], $p = 0,092$; $\Delta\text{chrF++} = +0,83$ [–0, 06, 1, 71], $p = 0,064$.
- **E2 – E3-custom:** $\Delta\text{BLEU} = +11,15$ [9, 26, 12, 77], $p = 0,002$; $\Delta\text{chrF++} = +9,46$ [8, 05, 10, 88], $p = 0,002$.

Lợi thế của E2 lớn và vững chắc về thống kê. Ở E3-custom–E1, cả hai khoảng tin cậy đều bắc qua 0 —không có ý nghĩa thống kê ở $\alpha = 0,05$, dù điểm ước lượng dương. Kết hợp với các yếu tố gây nhiễu khác giữa E1 và E3-custom đã nêu ở mục 3.5, mức tăng +0,99 BLEU của retrieval nên được hiểu là *chưa đo được một cách sạch sẽ*, không phải bằng chứng cho thấy retrieval không mang lại lợi ích.

4 Thực Nghiệm

Phần Phương pháp đã nói ba hệ thống được cấu hình như thế nào. Phần này nói chúng thực sự chạy ở đâu, và chạy theo thứ tự nào.

4.1 Computing Environment

4.1.1 Các thông tin về phần cứng

Cả ba experiment chạy trên cùng một máy với đúng một GPU: NVIDIA GeForce RTX 3060, bộ nhớ 12GB VRAM. Hệ điều hành là Linux-7.0.0-28-generic-x86_64-with-glibc2.39, Python 3.10.20 build bởi conda-forge.

4.1.2 Cách setup 3 môi trường cho 3 thí nghiệm khác nhau

Pipeline không chạy trong một môi trường duy nhất, chúng được chạy sử dụng trên 3 môi trường khác nhau để người chạy dễ audit hơn. Fairseq 0.12.2 chỉ chạy được với PyTorch cũ, còn kernel 4-bit của bitsandbytes thì chỉ có ở PyTorch mới. Hai yêu cầu này không thể ở chung, nên dự án tách làm ba (Bảng 6).

Bang 6: Ba môi trường Conda và phiên bản thực tế, đọc từ khối `packages` của các run manifest.

Môi trường	Dùng cho	Python / Py-Torch / CUDA	Thư viện chính
.conda/fairseq	E1, E3-custom	3.10.20 / 1.13.1 / 11.7	fairseq 0.12.2
.conda/llm	E2	3.10.20 / 2.5.1 / 12.4	transformers 4.57.6; peft 0.20.0; bitsandbytes 0.50.1; accelerate 1.14.0; datasets 4.8.5
.conda/eval	Chấm điểm, audit, test	3.10.20 / — / —	sacrebleu 2.6.0; PyYAML 6.0.2; pytest 8.4.1

Chỗ duy nhất ba môi trường gặp nhau là `sacrebleu` 2.6.0, và phiên bản này xuất hiện trong mọi metric signature đã lưu, dưới dạng `version:2.6.0`. Nhờ vậy, khác biệt giữa CUDA 11.7 và CUDA 12.4 chỉ ảnh hưởng lúc train, không đụng tới lúc chấm: cả ba hệ thống được đo bằng đúng một bản `sacrebleu`.

4.1.3 Seed và determinism

Để đảm bảo được việc các kết quả ở đây đều có thể chạy lại và cùng cho ra kết quả ban đầu, nhóm để cả ba config đặt `seed: 42`, và số này đi xuống mọi nguồn ngẫu nhiên mà pipeline với tới được. Phía Fairseq, trước khi gọi `fairseq-train`, pipeline bật `cuda.deterministic`, tắt `cuda.benchmark`, bật `use_deterministic_algorithms` ở chế độ `warn_only`, rồi truyền `CUBLAS_WORKSPACE_CONFIG=:4096:8` và `PYTHONHASHSEED` xuống các subprocess. Phía QLoRA, `set_seed` được gọi cả trước khi train lẫn trước khi sinh prediction; `TrainingArguments` nhận cả `seed` lẫn `data_seed`; bước packing xáo trộn bằng một generator riêng, cũng khởi tạo từ 42. Paired bootstrap ở phần đánh giá dùng seed 42 với 1.000 resample.

4.1.4 Thời gian chạy thực tế

Bảng 7 là thời gian train đo được. Theo chuẩn NMT hiện nay thì đây là những con số nhỏ, và đó là bối cảnh cần nhớ khi đọc phần Kết quả.

Bang 7: Tổng thời gian huấn luyện ghi nhận của 3 thí nghiệm

Hệ thống	Tổng wall-clock	Ghi chú
E1	≈ 3.209,5 s (≈ 53,5 phút)	Tổng lần chạy bị ngắt và lần chạy tiếp từ checkpoint tại update 579
E3-custom	4.758,7 s (≈ 79,3 phút)	Một lượt chạy, chạm <code>max_update</code> 50.000
E2	33.940,7 s (≈ 9,43 giờ)	Tổng hai đoạn huấn luyện, chạy tiếp từ checkpoint 546 đến step 910

Con số của E1 có một điều kiện đi kèm. Lần chạy được báo cáo không liền mạch: một lần trước đó bị ngắt giữa chừng, và checkpoint cuối là sản phẩm của lần thứ hai chạy tiếp từ epoch 5. Vậy 3.119,5 giây là chi phí của đoạn chạy tiếp, không phải chi phí dựng lại E1 từ đầu.

4.1.5 12 GB VRAM giải thích phần lớn config

Với E2, một mô hình 8 tỷ tham số không thể nạp ở fp16, nên weight nền bị quantize xuống NF4 kèm double quant và chỉ LoRA adapter nhận gradient. Kèm theo đó là gradient checkpointing, optimizer `paged_adamw_8bit`, batch mỗi device bằng 1 bù lại bằng accumulation 16 bước, và sequence window hạ từ 1.024 xuống 768 sau một lượt kiểm tra bộ nhớ. Với E1 và E3-custom, `max_tokens_per_gpu` bằng 1.000 kèm `update_freq` bằng 4 là cách lấy batch hiệu dụng khoảng 4.000 token mà vẫn vừa bộ nhớ.

Trần này cũng giải thích một bất đối xứng mà phần Phương pháp có nhắc. Khi E3-custom kéo source dài ra gần 4 lần vì phần hint Hán tự, cùng trần 1.000 token chứa được ít câu hơn hẳn, và batch thực tế rơi từ 132,5 xuống 33,3 câu mỗi update. Nói cách khác, E1 và E3-custom lệch ngân sách vì bộ nhớ, không phải vì ai đó chủ ý thiết kế như vậy.

4.1.6 Ba chỗ provenance còn hở

Ba điểm sau cần nói thẳng, thay vì gói chung vào một câu “thí nghiệm có thể tái lập”.

Thứ nhất, `git_revision` rỗng trong cả ba manifest. Có hash SHA-256 cho config, cho checkpoint và cho từng file corpus, nhưng không có gì ghi lại phiên bản code đã chạy. Nên không thể nói gì về khả năng tái lập ở mức source.

Thứ hai, log train gốc của đoạn E2 đầu tiên đã bị một lệnh `tee` ghi đè mất. Còn lại `training_history.json`, `trainer_state.json` và khối `train_metrics` trong manifest: đủ để đối chiếu số liệu, không đủ để soi lại từng dòng log.

Thứ ba là chuyện revision của model nền. Manifest ghi `model.revision` bằng `main`, đồng thời ghi `resolved_model_revision` bằng commit `b968826d9c46dd6066d109eabc6255188de91218`. Tức là lúc chạy, pipeline xin nhánh mặc định, rồi manifest mới ghi lại commit nó nhận được; còn bản config hard-pin đúng commit đó là bản hiện tại, không phải bản đã chạy. Đây chính là caveat “freeze record không khớp byte với config hiện tại” ở Bảng ?? . Commit sinh ra kết quả vẫn xác định và truy được, chỉ là nó được chốt *sau* khi chạy chứ không phải *trước*.

4.2 Experimental Design

4.2.1 So sánh cái gì với cái gì

Thực nghiệm đặt ba hệ thống dịch Việt → Hán văn cạnh nhau trên cùng ngữ liệu *Đại Việt sử ký toàn thư*: E1 là Transformer train from scratch, E2 là Qwen3-8B fine-tune bằng QLoRA, còn E3-custom là E1 với source được chèn thêm hint Hán tự lấy từ knowledge base.

Đây là so sánh cấp hệ thống, không phải một chuỗi ablation. Bảng ?? liệt kê trực nào giữ chung và trực nào khác nhau; Hình 1 cho thấy phần đầu và phần cuối dùng chung, kẹp ba recipe khác nhau ở giữa.

4.2.2 Ba split, và bằng chứng ba hệ thống dùng chung một corpus

Corpus chia cố định: 19.218 cặp train (silver, self-aligned), 510 cặp validation và 510 cặp test (đều gold, manually aligned). Train để cập nhật tham số, validation để chọn checkpoint, test chỉ mở sau khi lựa chọn đã freeze. Không target validation hay test nào lọt vào train.

Câu “ba hệ thống nhìn đúng cùng những hàng” không chỉ là cam kết trên giấy. Mỗi manifest ghi SHA-256 riêng cho từng phía của từng split, và cả sáu giá trị trùng nhau trên cả ba manifest, kèm

Bang 8: Tóm tắt setup thực nghiệm của ba hệ thống.

	E1	E2	E3-custom
Backbone	Transformer 6+6, $d = 512$, 67,65M tham số	Qwen3-8B nền đóng băng + LoRA $r = 16$, $\alpha = 32$	Như E1, source augmented, 74,64M tham số
Precision	fp16	Nền NF4 4-bit, double quant, tính toán fp16	fp16
Optimizer	Adam, 5×10^{-4} , inverse-sqrt, warmup 8.000	Paged AdamW 8-bit, 2×10^{-4} , cosine, warmup ratio 0,05	Như E1
Batch khai báo	1.000 token $\times$ accumulation 4	1 window $\times$ accumulation 16	Như E1
Ngân sách train	Trần 50.000 update / 500 epoch, patience 20; dừng ở epoch 171 / update 24.791	Ban đầu 3 epoch = 546 step; kéo lên 5 epoch = 910 step	Trần như E1; chạm 50.000 update ở epoch 87
Chọn checkpoint	Validation BLEU, epoch 151	Validation <code>eval_loss</code> , step 400	Validation BLEU, epoch 77
Decode	Beam 7; $1,2 x + 10$	Beam 4; $\min(512, \lceil 2 x \rceil + 32)$	Như E1

hash của knowledge base và dataset id `dvsktt_vi_zh_v1`. Corpus mà đổi giữa các lần chạy thì đã lộ ngay ở đây.

4.2.3 Tóm tắt ba setup

Bảng 8 gom ba recipe lại để đọc mà không phải lật ngược. Số liệu khớp với Bảng ?? và Bảng ??; chỗ khác duy nhất là hàng ngân sách của E2, sẽ nói ở Mục 4.2.6.

Hai cột batch không quy đổi được cho nhau: Fairseq đếm theo token mỗi GPU, còn E2 đếm theo số window đã pack. Đừng đọc hàng “Batch khai báo” như một so sánh về số câu mỗi update.

4.2.4 Validation trước, freeze, rồi mới mở test

Mỗi hệ thống đi đúng một trình tự: train, sinh prediction validation, chấm validation, freeze lựa chọn, rồi mới sinh prediction test. Bước freeze ghi lại hash của config, của checkpoint (hoặc adapter), của khối decoding, của prediction validation và của metric validation. Trước mỗi lần decode test, pipeline tính lại đúng những hash đó và dừng nếu lệch. Hình ?? mô tả cổng này.

Có thêm hai lớp kiểm tra nữa. Bộ chấm tự tính lại dạng canonical từ trường `prediction` rồi so với `prediction_scored` đã lưu, lệch là từ chối file, nên một output sửa tay hay canonicalize bằng code khác không lọt vào metric được. Bộ generate của Fairseq thì kiểm tra tập ID hypothesis đúng bằng 0 đến 509, nên mất dòng hay lặp dòng sẽ làm dừng chương trình chứ không âm thầm làm sai mẫu số.

Nhưng phải nói rõ: cổng này bảo vệ *quy trình*, chứ không làm cho tiêu chí giống nhau. E1 và E3-custom chọn checkpoint bằng BLEU nội bộ của Fairseq, còn E2 chọn bằng `eval_loss`. Chênh lệch cuối cùng vì thế mang theo cả cách chọn checkpoint, không riêng năng lực mô hình.

Việc kéo E2 lên 5 epoch còn để lại một vết trên giấy tờ. Run kết thúc thì cả thư mục adapter được ghi lại, nên hash cây thư mục trong freeze record không còn khớp, và cổng sẽ chặn mọi lần decode test tiếp theo cho tới khi freeze lại.

Cần tách hai chuyện. Trong thư mục adapter chỉ đúng hai file đối: `adapter_config.json` và `training_args.bin`, tức phần metadata ghi rằng run khai báo 5 epoch. File weight `adapter_model.safetensors`

Bảng 9: Toàn bộ các điểm đánh giá validation của E2 qua 910 step.

Step	Epoch	eval_loss	Thuộc đoạn
200	1,10	1,4444	3 epoch ban đầu
400	2,20	1,4322	3 epoch ban đầu — được chọn
600	3,30	1,4659	Phần kéo dài
800	4,40	1,5815	Phần kéo dài

giữ nguyên từng byte, và khối **decoding** của hai config giống nhau ở mọi trường. Nghĩa là những gì ảnh hưởng tới output đều không đổi; thứ đổi chỉ là metadata về ngân sách train.

Dù vậy, freeze record đã dùng để mở test và freeze record phải tạo lại sau này không cùng một chuỗi hash. Sợi dây buộc prediction test vào đúng bộ weight vẫn còn ở mức file weight, nhưng đã đứt ở mức freeze record. Báo cáo này không giấu chuyện đó.

4.2.5 Chấm điểm và kiểm định thống kê

Mọi prediction được đưa về cùng một dạng canonical rồi mới chấm bằng SacreBLEU và chrF2++, cấu hình chi tiết ở Bảng ???. Đơn vị so sánh là 510 **sample_id** ghép cặp, mỗi mẫu một reference. Độ bất định ước lượng bằng paired bootstrap: 1.000 lần lấy mẫu, seed 42. Mỗi lần rút 510 chỉ số có hoàn lại rồi tính lại toàn bộ corpus metric trên đúng tập chỉ số đó cho cả hai hệ thống. Cách ghép cặp này loại được phần biến thiên do hai hệ thống tình cờ gặp hai tập câu khó dễ khác nhau. Nó không xử lý được các confound về mô hình, cách train và cách chọn checkpoint đã liệt kê ở Bảng ???. Nó chỉ trả lời một câu hẹp hơn: chênh lệch quan sát được có trụ vững khi lấy mẫu lại tập test hay không.

4.2.6 E2: 3 epoch hay 5 epoch?

E2 ban đầu khai báo 3 epoch, tức 546 optimizer step. Sau đó nó được kéo lên 5 epoch, tức 910 step, bằng một run chạy tiếp từ checkpoint cũ. Vì **eval_steps** bằng 200, cả 910 step chỉ có bốn điểm đánh giá (Bảng 9).

eval_loss chạm đáy ở step 400 rồi tăng đều suốt phần kéo dài. Trainer được cấu hình **load_best_model_at_end** theo **eval_loss**, nên adapter xuất ra lúc kết thúc 910 step vẫn chính là checkpoint step 400. Chuyện này được kiểm chứng trực tiếp chứ không suy đoán: file weight của adapter và file weight trong thư mục checkpoint step 400 có cùng một SHA-256.

Vậy kết luận đúng là: kéo ngân sách từ 546 lên 910 step *không* tạo ra một hệ thống mới, mà cho thấy tiêu chí lựa chọn đã từ chối phần kéo dài đó. Mô hình bắt đầu overfit sau khoảng epoch 2,2, và mọi số liệu E2 ở phần sau đều ứng với checkpoint step 400.

Ba điều kèm theo. Bốn điểm đánh giá trên 910 step là một lưới rất thưa, nên đáy thật của đường validation có thể nằm lọt giữa hai điểm đo. Tiêu chí là **eval_loss** chứ không phải BLEU, nên đây là bằng chứng về hành vi của tiêu chí, không phải bằng chứng rằng 5 epoch dịch kém hơn. Và vì đoạn kéo dài chạy tiếp chứ không chạy độc lập, nó không phải một ablation ngân sách đúng nghĩa.

4.2.7 Những gì không nằm trong thực nghiệm này

Hai hệ thống trong spec gốc không có mặt ở đây. Bản E3 chính thức cần integration `knowledge_v2` mà repo không có, nên nó ở trạng thái blocked và backend chủ động dừng; hệ thống thực sự chạy là một biến thể riêng, và phải giữ hậu tố *custom* ở mọi bảng lần mọi diễn giải. E4, tức Qwen3 ghép cùng cơ chế retrieval của E3-custom, mới chỉ được implement và chạy preflight ước lượng chi phí, chưa từng train. Không số liệu nào của hai hệ thống này xuất hiện ở phần Kết quả.

4.3 Evaluation Metrics (Chỉ số đánh giá)

Để đánh giá chất lượng của các mô hình dịch máy từ tiếng Việt sang Hán văn cổ (Việt $\rightarrow$ Hán cổ), nghiên cứu này sử dụng hai chỉ số đánh giá tự động phổ biến: SacreBLEU và chrF++. Do ngữ liệu đích đã được phân tách bằng khoảng trắng ở cấp độ từng Hán tự, các chỉ số này chủ yếu phản ánh mức độ tương đồng ở cấp độ ký tự (character level).

4.3.1 4.3.1 SacreBLEU

Chỉ số BLEU (Bilingual Evaluation Understudy) đánh giá độ tương đồng giữa bản dịch do mô hình sinh ra và bản dịch tham chiếu dựa trên tỷ lệ trùng khớp của các n-gram (với $n \leq 4$).

- **Cấu hình thực tế:** Điểm BLEU-4 được tính bằng thư viện **SacreBLEU**, qua đó bảo đảm tính chuẩn hóa và nhất quán. Phiên bản được sử dụng trong kho mã nguồn là 2.6.0.
- **Chữ ký cấu hình (signature):** Cấu hình cụ thể của chỉ số BLEU được ghi lại bằng chữ ký tiêu chuẩn `nrefs:1|case:mixed|eff:no|tok:13a|smooth:exp|version:2.6.0`. Trong đó, phương pháp làm mượt là `exp` (exponential smoothing) và tùy chọn `effective_order` không được sử dụng.
- **Bộ tách từ:** Theo đặc tả, cả bốn nhóm thí nghiệm phải sử dụng thống nhất bộ tách từ Moses để bảo đảm độ tin cậy của phép so sánh giữa các hướng dịch. Trong **SacreBLEU 2.6.0**, cấu hình `tok:13a` tương ứng với bộ tách từ `mteval-v13a` của Moses. Do ngữ liệu Hán văn cổ đích đã được tiền xử lý bằng cách chèn khoảng trắng giữa từng Hán tự, bộ tách từ ít ảnh hưởng đến kết quả; vì vậy, điểm BLEU thu được về cơ bản phản ánh mức độ tương đồng ở cấp độ ký tự.

4.3.2 4.3.2 chrF++

Chỉ số chrF++ là phiên bản mở rộng của chrF, kết hợp độ tương đồng ở cấp độ ký tự (character-level similarity) với thông tin ở cấp độ từ (word-level information). Chất lượng bản dịch được đánh giá dựa trên độ chính xác (precision), độ bao phủ (recall) và điểm F (F-score).

- **Độ tương đồng ở cấp độ ký tự (character n-gram):** Chỉ số đo tỷ lệ trùng khớp của các n-gram ký tự có độ dài từ 1 đến 6 giữa bản dịch của mô hình và bản dịch tham chiếu. Thành phần này giúp ghi nhận các biến thể về từ vựng và cấu trúc trong Hán văn cổ.
- **Thông tin ở cấp độ từ (word n-gram):** Chỉ số bổ sung tỷ lệ trùng khớp của các n-gram từ có độ dài từ 1 đến 2. Trong ngữ liệu Hán văn cổ đã được phân tách bằng khoảng trắng, mỗi Hán tự được xem như một từ độc lập; do đó, thành phần n-gram từ bậc hai thực chất tương ứng với bigram Hán tự.

- **Precision, recall và F-score:** chrF++ sử dụng hệ số $\beta = 2$ để tính điểm F_β , tức là đặt trọng số cho recall cao hơn precision. Cách tính này nhấn mạnh khả năng bao phủ các đơn vị nội dung quan trọng trong bản dịch.
- **Chữ ký cấu hình (signature):** Chữ ký của chrF++ trong thực nghiệm là `nrefs:1|case:mixed|eff:y`.

4.3.3 COMET (tùy chọn)

Chỉ số COMET (Crosslingual Optimized Metric for Evaluation of Translation) sử dụng mô hình ngôn ngữ đa ngữ, chẳng hạn như XLM-RoBERTa, để biểu diễn câu nguồn, bản dịch tham chiếu và bản dịch của mô hình trong một không gian vector chung, từ đó đánh giá chất lượng dịch ở cấp độ câu.

- **Hạn chế khi áp dụng:** Mặc dù COMET thường có độ tương quan cao với đánh giá của con người đối với các ngôn ngữ hiện đại, một số checkpoint chủ yếu được huấn luyện trên dữ liệu thuộc các ngôn ngữ châu Âu. Vì khả năng bao phủ tiếng Việt và đặc biệt là Hán văn cổ có thể còn hạn chế, kết quả đánh giá ngữ nghĩa có nguy cơ thiếu chính xác hoặc kém nhạy. Do đó, COMET chỉ được xem là chỉ số tùy chọn và không được sử dụng làm thước đo chính trong các thực nghiệm của nghiên cứu này.

4.4 Quy trình đánh giá

Quy trình đánh giá được tổ chức thành một pipeline tự động, khép kín nhằm bảo đảm tính toàn vẹn của kết quả và ngăn ngừa rò rỉ dữ liệu (data leakage):

1. **Lựa chọn mô hình trên tập validation:** Mô hình được huấn luyện trên tập train và đánh giá định kỳ trên tập validation. Tiêu chí lựa chọn checkpoint khác nhau giữa các backend:
 - **Fairseq (E1/E3):** Checkpoint tối ưu được lựa chọn bằng cách tối đa hóa chỉ số BLEU trong quá trình huấn luyện; chỉ số này được tính bằng tùy chọn nội bộ `--eval-bleu`.
 - **Qwen3 QLoRA (E2):** Checkpoint tối ưu được lựa chọn bằng cách tối thiểu hóa giá trị hàm mất mát trên tập validation (`eval_loss`).
2. **Công đóng băng (freeze gate) bảo vệ tập test:** Sau khi xác định checkpoint tốt nhất trên tập validation, hệ thống thực thi lệnh đóng băng thực nghiệm (`freeze-selection`). Lệnh này tạo tệp `selection_frozen.json`, trong đó lưu mã băm SHA-256 của tệp cấu hình hệ thống, checkpoint mô hình và kết quả dịch trên tập validation. Quy trình giải mã và chấm điểm trên tập test chỉ được kích hoạt khi tệp đóng băng hợp lệ và các tham số mô hình cũng như tham số giải mã không bị thay đổi. Cơ chế này ngăn việc điều chỉnh các tham số giải mã, chẳng hạn như kích thước chùm (beam size) hoặc hệ số phạt (penalty), trực tiếp trên tập test nhằm tối ưu hóa điểm số.
3. **Chấm điểm trên tập test:** Quá trình suy luận trên tập test tạo ra bản dịch thô (`prediction_raw`); bản dịch này sau đó được làm sạch và chuẩn hóa khoảng trắng để tạo thành `prediction_scored`. Điểm BLEU và chrF++ được tính bằng thư viện SacreBLEU 2.6.0 thông qua việc đối sánh toàn bộ chuỗi `prediction_scored` với bản dịch tham chiếu đã được phân tách bằng khoảng trắng (`score_from_chinese(reference)`).

4. **Kiểm định thống kê bằng tái lấy mẫu bootstrap theo cặp:** Để đánh giá độ tin cậy của mức cải thiện điểm số giữa các thí nghiệm (ví dụ: E2 so với E1 hoặc E3 so với E1), hệ thống thực hiện tái lấy mẫu bootstrap theo cặp (paired bootstrap resampling):

- Lặp lại quá trình lấy mẫu có hoàn lại $N = 1,000$ lần với một seed cố định (`seed = 42`) để bảo đảm khả năng tái lập.
- Tính độ chênh lệch giữa các chỉ số (ΔBLEU và $\Delta\text{chrF++}$) trên mỗi mẫu bootstrap.
- Xác định khoảng tin cậy 95% từ các phân vị 2,5% và 97,5% của phân phối độ chênh lệch, đồng thời tính giá trị p hai phía để xác định liệu mức cải thiện có ý nghĩa thống kê hay không, với mức ý nghĩa $\alpha = 0,05$.

5 Kết quả

- Phát

5.1 Tổng quan

Phần này trình bày kết quả thực nghiệm của hai mô hình dịch máy chính trong dự án: mô hình cơ sở NMT Transformer (**E1**) huấn luyện từ đầu (from scratch) và mô hình ngôn ngữ lớn Qwen3-8B tinh chỉnh hiệu quả tham số bằng QLoRA (**E2**). Quá trình đánh giá được thực hiện trên tập kiểm định (Validation) và tập kiểm thử (Test) gồm 510 cặp câu tiếng Việt - Hán cổ. Điểm số BLEU và chrF++ được đo đạc ở cấp độ ký tự thông qua thư viện SacreBLEU 2.6.0.

Bảng 10 tổng hợp kết quả định lượng của các thí nghiệm trên cả hai tập dữ liệu.

Bảng 10: Kết quả định lượng chỉ số BLEU và chrF++ trên tập Validation và Test

Thí nghiệm	Val BLEU	Val chrF++	Test BLEU	Test chrF++
E1 (Fairseq Transformer)	29.37	31.12	28.63	30.58
E2 (Qwen3-8B QLoRA)	41.06	41.20	40.77	40.87
<i>E3-Custom</i> (Fairseq + Retrieval)*	31.04	32.72	29.62	31.41

*Lưu ý: *E3-Custom* là thí nghiệm bổ sung sử dụng phương pháp tra cứu tri thức nguồn và không phải phương pháp chính thức.

5.2 Kết quả thực nghiệm E1 (Fairseq Transformer)

Mô hình **E1** sử dụng kiến trúc Transformer cơ bản (6 lớp Encoder, 6 lớp Decoder, kích thước ẩn $d_{\text{model}} = 512$, tầng FFN = 2048, 8 đầu attention) được huấn luyện từ đầu trên tập Train gồm 19,218 cặp câu.

- **Quá trình huấn luyện:** Mô hình hội tụ và dừng sớm (early stopping) do vượt quá số epoch quy định của thuộc tính patience ($\text{patience} = 20$) tại epoch 171 (tương ứng với 24,791 bước cập nhật gradient). Checkpoint tối ưu nhất được lựa chọn ở epoch 151 dựa trên chỉ số BLEU lớn nhất trên tập Validation là 29.39 (đo trực tiếp khi train).

- **Hiệu năng tập Test:** Trên tập Test độc lập, mô hình E1 đạt điểm số BLEU là 28.63 và chrF++ là 30.58.
- **Đặc trưng thể hiện:** Mô hình E1 gặp hiện tượng sinh thiếu ký tự (under-generation). Chỉ số phạt độ dài câu ngắn (Brevity Penalty - BP) của E1 trên tập Test là 0.943 với tỷ lệ độ dài chuỗi dự đoán trên chuỗi tham chiếu thực tế là 8,486 / 8,982 ký tự. Điều này cho thấy mô hình NMT truyền thống khi huấn luyện trên tập dữ liệu nhỏ có xu hướng dịch ngắn hơn so với câu đích chuẩn để giảm thiểu rủi ro lỗi phạt n-gram.

5.3 Kết quả thực nghiệm E2 (Qwen38B QLoRa)

Mô hình **E2** thực hiện tinh chỉnh QLoRA trên mô hình nền tảng Qwen3-8B ở định dạng lượng tử hóa 4-bit NF4, sử dụng cấu hình tích hợp toàn bộ các tầng tuyến tính (**all-linear targets**) với rank $r = 16$ và hệ số scale $\alpha = 32$.

- **Quá trình huấn luyện:** Huấn luyện trong 3 epoch (tương đương 546 bước cập nhật với kích thước batch hiệu dụng là 16). Do tần suất đánh giá được cấu hình mỗi 200 bước, checkpoint tốt nhất được khôi phục tại bước 400 (epoch 2.2) với giá trị mất mát trên tập kiểm định thấp nhất ($\text{eval_loss} = 1.4322$), thay vì sử dụng checkpoint cuối cùng của epoch 3.
- **Hiệu năng tập Test:** Mô hình E2 cải thiện vượt bậc, đạt điểm số BLEU là 40.77 và chrF++ là 40.87 trên tập Test.
- **Đặc trưng thể hiện:** Mô hình E2 đạt điểm phạt câu ngắn tuyệt đối ($BP = 1.000$) với tỷ lệ độ dài chuỗi dự đoán gần như khớp hoàn toàn với câu đích tham chiếu (8,979 / 8,982 ký tự). Việc áp dụng kỹ thuật nhân mặt nạ chỉ tính loss trên câu trả lời (response-only loss) kết hợp cấu trúc prompt rõ ràng giúp LLM kiểm soát tốt hành vi sinh chuỗi dịch, tránh hiện tượng sinh lặp hoặc sinh thừa văn bản tiếng Việt.

5.4 So sánh đối chiếu E1 vs E2 (E1 vs E2 Comparison)

Bảng 11 phân tích độ chính xác n-gram ở cấp độ ký tự và hệ số phạt độ dài (BP) của hai mô hình trên tập Test để làm rõ nguồn gốc của sự cải tiến hiệu năng.

Bảng 11: So sánh chi tiết n-gram precision và Brevity Penalty (BP) trên tập Test

Mô hình	1-gram	2-gram	3-gram	4-gram	BP	Độ dài Dự đoán / Tham chiếu
E1	62.2	35.5	23.1	16.6	0.943	8,486 / 8,982
E2	69.9	46.6	33.6	25.3	1.000	8,979 / 8,982

- **Phân tích độ chính xác n-gram:** Điểm số cải tiến của E2 so với E1 tăng dần theo bậc của n-gram: chênh lệch tăng từ +7.7% ở 1-gram lên đến +8.7% ở 4-gram. Điều này chứng minh rằng mô hình LLM Qwen3 không chỉ cải thiện khả năng dịch đúng các ký tự Hán tự đơn lẻ mà còn vượt trội ở khả năng xây dựng các cụm từ ghép, cú pháp và cấu trúc ngữ pháp mạch lạc trong Hán văn cổ (phản ánh qua tỷ lệ khớp 4-gram cao vượt trội).

- **Kiểm định thống kê Paired Bootstrap:** Phép kiểm định tái lấy mẫu tự khởi động theo cặp ($N = 1,000$, $seed = 42$) trên tập Test cho thấy mức chênh lệch hiệu năng giữa E2 và E1 đạt ý nghĩa thống kê cực kỳ rõ rệt:
 - Độ chênh lệch BLEU trung bình (ΔBLEU) đạt **+12.14** với khoảng tin cậy 95% dao động trong khoảng **[10.22, 13.77]** và giá trị $p = 0.002$.
 - Độ chênh lệch chrF++ trung bình ($\Delta\text{chrF++}$) đạt **+10.29** với khoảng tin cậy 95% trong khoảng **[9.03, 11.79]** và giá trị $p = 0.002$.

Vì khoảng tin cậy 95% của cả hai độ đo đều nằm hoàn toàn phía trên điểm số 0 và giá trị p cực kỳ nhỏ ($p \ll 0.05$), giả thuyết không (null hypothesis) bị bác bỏ. Mô hình tinh chỉnh LLM (E2) cho thấy sự vượt trội thực sự về mặt thống kê so với mô hình Transformer truyền thống huấn luyện từ đầu (E1).

5.5 Phân tích ví dụ định tính (Qualitative Examples)

Mặc dù quy trình dán nhãn lỗi thủ công 300 mẫu (error analysis sheet) của dự án vẫn đang trong quá trình thực hiện độc lập bởi các thành viên nhóm, các phân tích sơ bộ dựa trên đặc tính cấu trúc cho phép nhận diện một số xu hướng lỗi đặc trưng của hai mô hình:

1. Lỗi bỏ sót thông tin (Omission):

- *Đặc trưng E1:* Do xu hướng dịch ngắn ($\text{BP} = 0.943$), E1 thường bỏ sót các thành phần trạng ngữ chỉ thời gian hoặc các hư từ (particles) trong Hán văn cổ. Ví dụ, các từ nguồn chỉ thời gian như “năm sau”, “mùa đông” đôi khi bị mô hình bỏ qua để tập trung dịch các động từ và danh từ chính.
- *Đặc trưng E2:* Lỗi bỏ sót xuất hiện rất ít nhờ khả năng bao phủ ngữ nghĩa tốt của mô hình ngôn ngữ lớn đã được huấn luyện trên ngữ cảnh dài.

2. Lỗi thực thể (Entity Error):

- *Đặc trưng E1:* Gặp khó khăn lớn đối với các tên người, tên chức vị cổ hoặc địa danh hiếm (ví dụ: các chức quan như “Đô tổng binh”, hoặc địa danh cổ). E1 dễ dịch sai lệch ký tự do các từ này xuất hiện với tần suất cực thấp trong tập Train và không có tri thức ngoại vi hỗ trợ.
- *Đặc trưng E2:* Thể hiện thế mạnh vượt trội đối với thực thể nhờ tri thức nền (world knowledge) khổng lồ tích lũy từ quá trình tiền huấn luyện (pre-training) của Qwen3 trên các văn bản cổ và tài liệu Hán ngữ cổ phong phú, giúp mô hình ánh xạ chính xác tên thực thể từ phiên âm Việt sang đúng ký tự Hán cổ tương ứng.

3. Lỗi dịch sai (Mistranslation) và sinh từ ngoài ngữ cảnh (Unsupported Addition):

- *Đặc trưng E1:* Lỗi dịch sai thường xuất hiện dưới dạng các ký tự không xác định ($\langle\text{unk}\rangle$) do gặp phải các từ vựng ngoài từ điển (OOV) hoặc các ký tự Hán tự hiếm không vượt qua được ngưỡng lọc từ vựng khi huấn luyện.

- *Đặc trưng E2*: Không gặp lỗi <unk>, tuy nhiên mô hình sinh chữ đôi khi có xu hướng “ảo tưởng” (hallucination) hoặc tự ý thêm các từ mang tính giải thích ngữ nghĩa khi gặp những câu nguồn tiếng Việt quá ngắn hoặc có cấu trúc mơ hồ. Lỗi này thường được khắc phục hiệu quả bằng cách điều chỉnh nhiệt độ giải mã (**temperature**) về mức cực thấp (giải mã tham lam/greedy-free beam search).

6 Error Analysis

6.1 Phương pháp phân tích (Error Analysis Methodology)

Để đánh giá toàn diện những hạn chế mà điểm số tự động không phản ánh được, nhóm nghiên cứu thực hiện quy trình dán nhãn lỗi thủ công đối với các mẫu dự đoán.

Thông qua phân tích sơ bộ dựa trên đặc tính cấu trúc, phương pháp này đã nhận diện được các xu hướng lỗi đặc trưng của từng hệ thống dịch.

6.2 Hệ thống phân loại lỗi (Error Taxonomy)

Dựa trên đặc điểm của bài toán dịch từ tiếng Việt sang Hán văn cổ, hệ thống phân loại chú trọng vào các nhóm lỗi sau:

1. CORRECT (Chính xác): Bản dịch chính xác về mặt ngữ nghĩa; cách diễn đạt có thể khác biệt so với bản tham chiếu duy nhất (single reference). Nhãn này không thể xuất hiện đồng thời cùng với bất kỳ nhãn lỗi nào khác.
2. MISTRANSLATION (Dịch sai): Nội dung câu nguồn bị biểu diễn và truyền tải sai lệch, ngoại trừ các trường hợp đã được xếp vào các phân loại cụ thể bên dưới.
3. OMISSION (Bỏ sót): Bản dịch lược bỏ hoặc thiếu vắng các thông tin có ý nghĩa từ văn bản nguồn.
4. UNSUPPORTED_ADDITION (Sinh từ ngoài ngữ cảnh): Bản dịch đưa vào các thông tin không được hỗ trợ hoặc không có cơ sở từ câu nguồn. Lưu ý không sử dụng nhãn này nếu đó chỉ đơn thuần là một cách diễn đạt lại (paraphrase).
5. ENTITY_ERROR (Lỗi thực thể): Dịch sai hoặc nhầm lẫn tên người, địa danh, triều đại, chức tước, tổ chức hoặc các thực thể có tên riêng (named entity).
6. NUMBER_OR_TIME_ERROR (Lỗi thời gian/số lượng): Dịch sai về số lượng, ngày tháng, niên hiệu, khoảng thời gian, trình tự sự kiện hoặc các mối quan hệ thời gian.
7. OTHER (Lỗi khác): Một vấn đề khác không được bao hàm trong các nhóm trên. Các trường hợp chọn nhãn này được giải thích rõ ràng trong phần ghi chú.

6.3 Phân bố lỗi (Error Distribution)

Bảng dưới đây trình bày chi tiết sự phân bố tuyệt đối (tần suất) và tương đối (tỷ lệ phần trăm) của từng nhóm lỗi trên tổng số 100 câu kiểm tra:

Bang 12: Phân loại lỗi của các mô hình

Nhãn Phân Loại Lỗi	Mô hình e1 (Fairseq)	Mô hình e2 (Qwen3-8B QLoRA)	Mô hình e3 (Fairseq + Retrieval)
CORRECT	3 (3.0%)	7 (7.0%)	4 (4.0%)
MISTRANSLATION	58 (58.0%)	63 (63.0%)	53 (53.0%)
OMISSION	23 (23.0%)	15 (15.0%)	19 (19.0%)
UNSUPPORTED_ADDITION	11 (11.0%)	10 (10.0%)	14 (14.0%)
ENTITY_ERROR	0 (0.0%)	0 (0.0%)	0 (0.0%)
NUMBER_OR_TIME_ERROR	18 (18.0%)	9 (9.0%)	23 (23.0%)
OTHER	0 (0.0%)	0 (0.0%)	0 (0.0%)

Từ bảng phân bố lỗi trên, cấu trúc phân bố của từng mô hình thể hiện các đặc trưng riêng biệt:

- Mô hình e1 (Baseline Fairseq): Có độ lệch lớn về lỗi bỏ sót thông tin (OMISSION chiếm tới 23.0%), cho thấy giới hạn của kiến trúc encoder-decoder dung lượng nhỏ khi gặp các câu văn lịch sử dài dòng, nhiều mệnh đề phụ.
- Mô hình e2 (Qwen3-8B QLoRA): Cải thiện rõ rệt tỷ lệ câu đúng (CORRECT đạt 7.0%, cao nhất) và kéo giảm mạnh lỗi thiếu sót (OMISSION chỉ còn 15.0%) cùng lỗi số liệu (NUMBER_OR_TIME_ERROR chỉ còn 9.0%). Tuy nhiên, mô hình này lại có tỷ lệ MISTRANSLATION cao nhất (63.0%) do thói quen chọn từ vựng hiện đại thay vì từ cổ văn thuần túy.
- Mô hình e3 (Knowledge-Retrieval): Việc tích hợp truy xuất giúp hạ thấp tỷ lệ MISTRANSLATION xuống mức 53.0% (tốt nhất trong 3 mô hình), nhưng lại đẩy tỷ lệ lỗi số liệu (NUMBER_OR_TIME_ERROR lên tới 23.0%) và ảo giác (UNSUPPORTED_ADDITION lên 14.0%) lên mức cao nhất do hiện tượng nhiễu ngữ cảnh từ kho tri thức bên ngoài.

6.4 So sánh đối chiếu lỗi (Error Comparison)

Dựa trên dữ liệu phân bố, sự khác biệt về kiến trúc đã tạo ra những xu hướng lỗi đặc trưng:

- Mô hình e2 (Qwen3-8B QLoRA): Thể hiện sự vượt trội về độ mượt cú pháp nhờ năng lực tổng quát hóa của LLM nền tảng, giúp giảm thiểu đáng kể tình trạng cắt câu hay bỏ sót ý (OMISSION). Dù vậy, điểm yếu của e2 vẫn nằm ở việc lạm dụng từ vựng Hán ngữ hiện đại, dẫn đến tỷ lệ MISTRANSLATION lớn nhất.
- Mô hình e1 (Fairseq cơ bản): Bộc lộ nhiều điểm nghẽn ở khả năng duy trì mạch thông tin dài, dẫn đến việc thường xuyên cắt xén hoặc bỏ quên các thành phần định ngữ phức tạp của câu nguồn.
- Mô hình e3 (Truy xuất tri thức): Giải quyết tốt hơn các bài toán tra cứu từ vựng cố định, nhưng lại thiếu cơ chế kiểm soát nội dung, khiến các con số thống kê, niên hiệu dễ bị sai lệch hoặc sinh thêm các chi tiết ngoài ngữ cảnh.

6.5 Hiện tượng Ảo giác (Hallucination)

Phân tích định tính sâu hơn về hiện tượng Ảo giác (UNSUPPORTED_ADDITION) cho thấy sự khác biệt rõ rệt về bản chất sinh lỗi giữa mô hình LLM (e2) và mô hình truyền thống kết hợp truy xuất (e3):

- Ảo giác nội sinh ở Mô hình LLM (e2): Xuất phát từ quá trình giải mã khi mô hình cố gắng đoán nghĩa và tự ý thêm các từ đệm, trạng từ hoặc thay đổi cấu trúc để câu trông trôi chảy hơn theo văn phong quen thuộc trong giai đoạn tiền huấn luyện, dù ý đó không có trong nguồn.
- Ảo giác ngoại sinh ở Mô hình Truy xuất (e3): Đến từ sự bất định của module truy xuất bên ngoài. Nếu văn bản nguồn chứa các sự kiện trùng lặp tên nhân vật ở nhiều triều đại, bộ truy xuất dễ lấy nhầm ngữ cảnh của sự kiện khác, khiến mô hình bị ép dịch theo ngữ cảnh nhiễu và tự ý chèn thêm các nhân vật, niên hiệu xa lạ.

6.6 Đề xuất

- Mở rộng kho từ vựng chuyên ngành (Domain-specific Gazetteer): Xây dựng bộ từ điển tra cứu nhanh cho các thuật ngữ hành chính, chức quan và địa danh lịch sử nhằm hạn chế hiện tượng dùng từ hiện đại (MISTRANSLATION).
- Lọc nhiễu cơ chế truy xuất tri thức (Strict Retrieval Filtering): Thắt chặt thuật toán chấm điểm độ tương đồng ngữ cảnh cho mô hình e3 nhằm ngăn chặn hoàn toàn hiện tượng nhiễu ngoại biên gây ảo giác.
- Ràng buộc từ vựng giải mã (Constrained Decoding): Áp dụng các kỹ thuật ràng buộc từ vựng miền để kìm hãm xu hướng dùng từ hiện đại của LLM, bảo đảm văn phong đầu ra đạt chuẩn mực cổ văn lịch sử.

7 Thảo luận, Hạn chế, kết luận

- Vũ, Phát

7.1 Thảo luận

- Kết quả thực nghiệm cho thấy sự vượt trội hoàn toàn của phương pháp tinh chỉnh mô hình ngôn ngữ lớn (E2) so với mô hình NMT truyền thống huấn luyện từ đầu (E1).
- Với mức chênh lệch trung bình lên tới +12.14 điểm BLEU và +10.29 điểm chrF++, E2 đã chứng minh rằng tri thức tiền huấn luyện (pre-training) đóng vai trò cốt lõi trong việc xử lý ngôn ngữ tài nguyên thấp như Hán văn cổ.
- Trong khi mô hình E1 chệch vật với các Hán tự hiếm và sinh ra lỗi <unk> do thiếu hụt dữ liệu huấn luyện, E2 có khả năng ánh xạ cực tốt các thực thể lịch sử từ phiên âm tiếng Việt sang Hán tự nhờ vào lượng tri thức nền (world knowledge) khổng lồ.

- Tuy nhiên, sự khác biệt về kiến trúc cũng mang lại các hành vi lỗi trái ngược: NMT truyền thống (E1) có xu hướng dịch an toàn bằng cách lược bỏ thông tin (under-generation), trong khi LLM (E2) lại dễ rơi vào trạng thái ảo giác (hallucination), tự ý bổ sung ngữ nghĩa khi gặp câu nguồn mơ hồ.

7.2 Hạn chế

Nghiên cứu vẫn tồn tại một số giới hạn nhất định về mặt dữ liệu, kỹ thuật và thiết kế thực nghiệm:

- Hạn chế về dữ liệu: Ngữ liệu huấn luyện có quy mô nhỏ (19.218 cặp câu) và chỉ khu trú trong một tác phẩm duy nhất là Đại Việt sử ký toàn thư, do đó mô hình có thể gặp khó khăn khi tổng quát hóa sang các thể loại văn bản cổ khác như thơ, phú hay chiếu biểu.
- Chất lượng tập huấn luyện: Tập Train là dữ liệu silver (cần chỉnh tự động) nên vẫn chứa nhiễu, đồng thời tồn tại khoảng 8,3% số dòng là các cặp câu trùng lặp, làm giảm lượng thông tin độc lập và có thể khiến mô hình bị thiên lệch.
- Thiếu ngữ cảnh tài liệu: Các câu được dịch một cách độc lập, làm mất đi các tín hiệu ngữ cảnh quan trọng cần thiết để giải thích đại từ tình lược, chức danh hay niên hiệu trong sử liệu.
- Hạn chế của chỉ số đánh giá: Tập kiểm định và kiểm thử chỉ cung cấp một câu tham chiếu duy nhất cho mỗi câu nguồn, điều này có thể khiến các chỉ số n-gram như BLEU đánh giá thấp các bản dịch có cách diễn đạt khác nhưng vẫn đúng nghĩa.
- Ràng buộc phần cứng và khả năng tái lập: Giới hạn bộ nhớ VRAM 12GB đã chi phối phần lớn các cấu hình siêu tham số (như kích thước batch và giới hạn token). Bên cạnh đó, việc huấn luyện E1 từng bị ngắt quãng và phải chạy tiếp nối, cũng như sự sai lệch mã băm (hash) trong hồ sơ đóng băng (freeze record) của E2 khi tăng số epoch, là những điểm yếu ảnh hưởng đến khả năng tái lập hoàn hảo chỉ bằng một lệnh chạy duy nhất.
- Sự bất cân xứng trong thiết kế đối chứng: Việc so sánh giữa E1 và biến thể E3-custom chưa thực sự hoàn hảo do sự chênh lệch về kích thước mô hình, chiều dài đầu vào và số lượng câu thực tế trong mỗi batch.

7.3 Kết luận

- Nghiên cứu đã hoàn thành mục tiêu xây dựng và đối chiếu thành công hai hệ thống dịch máy từ tiếng Việt sang Hán văn cổ trong điều kiện dữ liệu giới hạn.
- Trả lời cho các câu hỏi nghiên cứu, có thể khẳng định rằng việc áp dụng kỹ thuật QLoRA để tinh chỉnh mô hình ngôn ngữ lớn (Qwen3-8B) mang lại hiệu quả vượt xa so với việc huấn luyện mô hình Transformer từ đầu.
- Phương pháp LLM không chỉ cải thiện mạnh mẽ các chỉ số đánh giá tự động mà còn giải quyết xuất sắc bài toán dịch thực thể lịch sử - một trong những thách thức lớn nhất của miền văn bản cổ.

- Mặc dù còn những tồn đọng liên quan đến hiện tượng ảo giác của LLM và các giới hạn về tài nguyên phần cứng, đề án đã cung cấp một quy trình thực nghiệm rõ ràng, mã nguồn đóng băng chặt chẽ và một hệ thống phân loại lỗi đặc thù, tạo tiền đề vững chắc cho các nghiên cứu xử lý ngôn ngữ lịch sử trong tương lai.

Tài liệu

- [1] Alex R. Atrio and Andrei Popescu-Belis. Small batch sizes improve training of low-resource neural MT. In *Proceedings of the 18th International Conference on Natural Language Processing*, pages 18–24, 2021.
- [2] CLC Laboratory. Project specification: Machine translation for ancient texts, version 1.3, 2026. Tài liệu đặc tả nội bộ của đề án.
- [3] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient fine-tuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [4] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- [5] Myle Ott, Sergey Edunov, Alexei Baevski, Angela Fan, Sam Gross, Nathan Ng, David Grangier, and Michael Auli. fairseq: A fast, extensible toolkit for sequence modeling. In *Proceedings of NAACL-HLT 2019: Demonstrations*, pages 48–53. Association for Computational Linguistics, 2019.
- [6] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: a method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, 2002.
- [7] Maja Popović. chrF: Character n-gram F-score for automatic MT evaluation. In *Proceedings of the Tenth Workshop on Statistical Machine Translation (WMT)*, pages 392–395, 2015.
- [8] Maja Popović. chrF++: Words helping character n-grams. In *Proceedings of the Second Conference on Machine Translation*, pages 612–618, 2017.
- [9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.